Radical Denesting with `Strad.wl`

Third unified analysis: three reviews of the second corrected version and the third corrected version `StradFixed3.wl`

Analysis session for Vladimir Reshetnikov
`RadicalDenest` repository, `src/code-review/unified-C`

5 September 2026

Abstract

The second corrected version of the radical denester, `StradFixed2.wl`, was reviewed three times (`src/code-review/review-7`, `review-8`, `review-9`). Each review pinned the same commit, read the complete source, wrote a proposed `StradFixed3.wl` and a native test suite that it could not run, and executed SymPy checks of the mathematics it relied on. This document compares the three reviews, consolidates their findings with the observations of this session into a catalogue of 23 issues, and presents `src/corrected/StradFixed3.wl`, which addresses them. The main changes are a sound admission test for `Root` and `AlgebraicNumber` objects; an equality classifier that is exact in every branch, with the numeric prefilter reduced to optional pruning inside the search; incumbents threaded through every proposal stage; index reductions offered before recursion; a budget-aware memo that never lets a weak recursive search suppress a stronger one; every kernel operation, the classification of the input and the report costs inside the resource region; a discriminant batch cap enforced in the inner loop; malformed calls rejected with a `Failure`; recursive cost accounting; labelled certificate records; and three coverage extensions: the trace–norm criterion for every odd index up to a bound, a Honsbeek recognizer that accepts every term with a rational cube, and a square-class coset search for multi-surd square roots that closes the gap all three reviews proved.

Everything reported here was executed in Wolfram 15.0.1. The unified-A battery (116 rows, now with 33 round-3 cases) and fourteen randomized families (430 inputs) produce no wrong value; the fifteen inputs of `KNOWN_GAPS.md` stay denested; the 230 regression tests, which translate the three reviews' suites, pass; the reviews' own native suites were executed against their own proposals (Section 6.7); and review-8's differential corpus was run against both versions. Remaining limitations are stated in Section 7.

Evidence. All executions were performed in *15.0.1 for Microsoft Windows (64-bit) (July 2, 2026)* through `wolframscript`, one kernel at a time. Every table is generated mechanically from the recorded runs in `logs/` by `harness/export_tables3.wl`. Statements about the reviews cite their finding identifiers; statements about `StradFixed2.wl` cite the second unified analysis in `src/code-review/unified-B` ("unified-B" below). The reviewed file is `StradFixed2.wl` at commit `9000d6f`, Git blob `0bdeded3`, SHA-256 `e2a885da\ldots` ; the new file `StradFixed3.wl` has SHA-256 `ee647bee\ldots` .

Contents

1 Scope, material, and method

1.1 The object under analysis

`src/corrected/StradFixed2.wl` (745 lines, context `RadicalDenest2`‘) is the second corrected version of `src/original/Strad.wl`, produced in unified-B. It has one acceptance gate for every candidate, a session with a shared deadline and bounded kernel operations, validated options, a congruence walk over arithmetic hosts, exact fast paths for the quadratic, Gaussian, cubic-in-quadratic, Honsbeek and multi-surd square roots, negative-radicand separation, Kummer linear factors and index reduction, and a capped multiplier queue. Unified-B evaluated it on 116 battery rows and 340 randomized inputs with no wrong value, and on the fifteen previously missed inputs, all denested.

Three independent reviews of it were written on 5 September 2026; they are kept under `src/code-review/` as `review-7`, `review-8` and `review-9` and are called R7, R8 and R9 below. Unlike the previous round, all three could read the complete source at a pinned commit (Table 1). None could run a Wolfram kernel; each says so and supplies native probes, a native suite and a proposed implementation for execution here.

	R7	R8	R9
Title	A second-generation audit of `StradFixed2`	Radical denesting after the second repair	A contract-oriented review of `StradFixed2`
Source seen	complete, pinned `9000d6f`, blob checked	complete, web-rendered views of the pinned commit	complete, pinned `9000d6f`, blob checked
Findings	F01–F12	F01–F14	F1–F10
Executed checks	14 190 SymPy assertions	3 412 SymPy, control-flow and structural checks	396 SymPy and structural checks
Wolfram execution	none	none	none
Proposed code	816 lines, 36 checked edit groups	613 lines, a rewrite with eight new options	875 lines, four new options
Native tests supplied	50	52	40
Distinctive content	Dickson recurrences for every odd index; Honsbeek test family; cost of the transcendental `Root`	coset enumeration (`cosetBases`, `surdSystem`); opt-in list and inexact-host policies	character reconstruction over $\mathbb{F}_2^r$; discriminant batch loop model (24 nominal, 46 actual); differential corpus

Table 1: The three reviews of `StradFixed2.wl` at a glance.

1.2 What this document adds

1. A finding-by-finding comparison of the three reviews, with an assessment of each finding against the source and, where the finding is about executable behaviour, against a kernel run of the review’s own probe (Section 2).
2. A consolidated catalogue of 23 issues C01–C23, each with its sources, its disposition in `StradFixed3.wl` and, where the new version departs from a review’s recommendation, the reason (Section 3).
3. The design changes of `StradFixed3.wl`, read by responsibility (Section 4), and the mathematics of the three coverage extensions (Section 5).
4. Executed experiments: the battery of unified-A/B and a group of round-3 cases on both versions side by side, fourteen randomized families, the fifteen previously missed inputs, a regression

suite of 230 tests, the reviews' own suites against their own proposals, and review-8's differential corpus (Section 6).

5. Remaining limitations, compatibility changes, and future work (Sections 7 and 8).

2 The three reviews

2.1 Where they agree

The three reviews converge to a degree the previous round did not: eight of their findings are shared by all three, and every review states explicitly that it found no wrong value returned by `StradFixed2.wl` on an admitted, genuinely algebraic, parameter-free input. Their shared findings are the following.

- **The admission test is not sound** (R7 F01, R8 F01, R9 F1). `ExactAlgebraicQ` accepts any `Root` or `AlgebraicNumber` object of infinite precision, so `Root[##^5 + ## - Pi &, 1]` and `Root[##^3 - ## + parameter &, 1]` pass; exactness of a representation is not algebraicity. Confirmed in the kernel (Section 2.4). No wrong value follows, because a non-algebraic "island" can never be certified equal to a radical candidate, but the public predicate lies and `EqualityStatus` of such an object could in principle be decided by `RootReduce` in an undefined way.
- **A numeric hint produces the public status `"Different"`** (R7 F02, R8 F05, R9 F7). `certify` rejects a difference of reported accuracy at least 25 digits before the exact steps, so `EqualityStatus` is not the exact classifier its usage string describes, and `CertificatesDifferent` counts heuristic rejections. Unified-B had documented the prefilter as a heuristic rejection; the reviews are right that a public exact status cannot rest on it. Confirmed (Section 2.4).
- **Later stages reset the incumbent** (R7 F03, R8 F02, R9 F2). In `powerProposals` the lines `best = negativeRadicandCandidate[target, rho, p, q]` and `best = kummerCandidates[target, rho, p, q]` assign the stage result, and both helpers start from `target`, so a cheaper incumbent found by the fast paths is lost when the later stage finds nothing. Confirmed with R8's probe (Section 2.4). The public pipeline usually recovers the value through a later stage, which is why no battery row showed it.
- **Index reduction demands recursive progress** (R7 F04, R8 F03, R9 F3). `kummerCandidates` offers $\gamma^{1/(q/k)}$ only if the recursive call changes it *and* lowers its depth, so $(3+2\sqrt{2})^{1/4} \to \sqrt{1+\sqrt{2}}$ and $(3+2\sqrt{2})^{1/6} \to (1+\sqrt{2})^{1/3}$, which are cheaper without being shallower, depend on a different route. Confirmed: with `recurse` replaced by the identity, the helper returns the input (Section 2.4).
- **The memo caches failures without their budget** (R7 F06, R8 F04, R9 F4). An island first met inside a recursive call, which has a quarter of the trials, is memoized as unchanged and the memo answers a later top-level visit with the full budget.
- **Some work escapes the resource wrappers** (R7 F05, R8 F06, R9 F5). `FactorInteger` of a discriminant, `Together` in `linearRoots`, `Expand` of the multiplier ansatz and the fast-path dispatcher run outside `bounded[]`; the classification of the input and the two `RadicalCost` calls of the report run outside the `TimeConstrained` region, so a report can spend unbudgeted time and a disabled call still computes costs.
- **Malformed calls stay unevaluated** (R7 F07, R8 F14, R9 F8). `Strad[Sqrt[2], 17]` and `Strad[]` do not match `OptionsPattern[]` and return unevaluated instead of the documented `Failure`. Confirmed.

- **The multi-surd solver misses square-class cosets** (R7 F10, R8 F07, R9 F10). The two unknown sets $\{1\} \cup P$ and $\{1\} \cup P \cup D$ cannot express $\sqrt{6}+\sqrt{35}+\sqrt{77}$, whose square is $118+2\sqrt{210}+14\sqrt{55}+2\sqrt{462}$, because the support $\{6, 35, 77\}$ is a coset of the group generated by the radicands. All three reviews prove this; R9 proves further that closing the prime basis does not suffice. Confirmed: the fast path misses, and in unified-B the identity was found by the multiplier search in about five seconds (Section 2.4).

2.2 Findings unique to one or two reviews

R9 F6 models the discriminant batch of `multiplierSearch`: the check `batch >= 24` sits in the outer loop over primes, so with eight primes and $q = 7$ one prime can push the batch to 46 proposals before the check. The global admission cap holds, so the effect is a quota overrun, not an unbounded loop. R8 F12 makes the same observation and adds that multiplier keys are syntactic, so $2\sqrt{2}$ and $\sqrt{8}$ count as different multipliers. R7 F08 notes that `FastPathAccepted` is incremented when the fast paths *proposed* something, not when a proposal was accepted; R7 F09 and R8 F13 ask that the `Certificates` list be labelled as a truncated sample of accepted proposals rather than a proof chain of the final result, and that the result-changed fact be separated from the termination status. R7 F11 and R9 F9 examine the cost: `bitSize` prices integers and rationals but not the components of a `Complex`, so $1 + 2^{100}i$ costs the same as $1 + i$; and `radicalNodes` subtracts a count inside opaque payloads from a global count, which is brittle when payloads overlap. Confirmed for the Gaussian case (Section 2.4).

R8 F08 shows that the Honsbeek recognizer of unified-B is narrower than the identity it implements: the term pattern requires two cube-root terms of the form $c\,r^{1/3}$, so $\sqrt{28^{1/3}-3}$, whose second term is a rational, and $\sqrt{5^{1/3}-4^{1/3}}$, whose second term the kernel writes as $2^{2/3}$, are not recognized (both are found by the multiplier search instead). R8 F09 and R7 (Section 6 of its report) derive the trace–norm criterion for every odd index through the Dickson polynomials and propose it as a fast path; R9 verifies the cubic case and states the general criterion in its mathematical audit. R8 F10 and F11 are policy proposals: an independent progress test for list elements and processing of exact islands inside inexact hosts, both opt-in. R9's Section 9 proposes a character-reconstruction kernel over $\mathbb{F}_2^r$ for the multi-surd gap, exponential in the square-class rank r, capped at rank 2 by default.

2.3 Claims that we weigh differently

- **No negative caching at all.** All three reviews want the memo to store improvements only. We adopt the diagnosis, not the remedy: with no negative memo, the recursive index-reduction search re-runs the same unchanged sub-problem several times per island (the values $\gamma\zeta_k^j$ for every γ and j, the even-index split, and the multiplier search's equal-depth candidates all recurse into the same few islands). In a development run of `StradFixed3.wl` with a positive-only memo, $(3+2\sqrt{2})^{1/4}$ took 64 s and $(3+2\sqrt{2})^{1/6}$ 85 s, and with `"MaxRecursion" -> 1` the quartic case exhausted the 120 s budget and returned its input. The memo of `StradFixed3.wl` therefore stores, with every result, the budget of the search that produced it (the trials and recursion levels available) and whether it completed; a stored negative is reused only by a search with no more budget than the stored one. This gives the reviews' guarantee that a weak recursive search never suppresses a stronger top-level one (tested), at the cost they were worried about only when the budgets are equal (C05).

- **The numeric prefilter.** The reviews want it off by default and diagnostic only. We agree on both, and in addition remove it from `certify` altogether: it now lives in the search gate `accept`, before the exact certificate, where a rejection can lose a candidate but cannot produce a status. Measured on the battery, the default `False` costs nothing measurable: the eight slowest rows of the battery, timed with the prefilter off and on (log `prefilter_timing.txt`), show

identical certificate counts and zero numeric rejections, because every candidate that reaches the exact certificate has already passed the cost comparison, which rejects the wrong-branch orbit candidates first; the switch is kept for callers whose custom solver produces many expensive equal-cost candidates (Section 6.9).

- **The coset search.** R8 implements the coset enumeration but leaves it off by default (`"SquareClassSearch" -> False`) because of the cost of the rational systems; R9 implements a character transform capped at rank 2. We enable the coset search by default but make it cheap: for each coset an integer relation between $\sqrt{\rho}$ and the square roots of the coset is looked for numerically with `FindIntegerNullVector`, and the resulting candidate is kept only if its square reduces exactly to ρ. The rational `Solve` systems remain as a fallback. The four three-surd examples of the reviews are found in 0.3–0.6 s each (Table 8); the previous version needed the multiplier search and 2–17 s.
- **Optional list and inexact-host policies (R8 F10, F11).** Not adopted. Both are alternative semantics rather than defects; the previous round considered and declined the inexact-host processing, and an "independent" list progress that accepts a cheaper element while the whole list becomes costlier contradicts the single cost order the rest of the design relies on. Both remain documented policies (C16).
- **Canonical multiplier keys (R8 F12).** Not adopted: the syntactic key was chosen in unified-B precisely because `RootReduce` of every proposed multiplier dominated the search time; duplicates cost one dictionary lookup each (C15).
- **Character reconstruction (R9).** The coset enumeration of R8 and the character transform of R9 search the same space (the single-coset theorem, Section 5.2); the character transform needs 2^r conjugate square roots and up to 2^{2^r-1} sign patterns, whereas an integer relation per coset needs one numeric vector of length $|H|+1$. We adopt the coset enumeration with the relation stage (C17).

2.4 Kernel reproductions of the reviews' probes

The reviews' probes against `StradFixed2.wl` were run (`harness/probe_reviews2.wl`, log `logs/probe_reviews2.txt`): `ExactAlgebraicQ[Root[##^5 + ## - Pi &, 1]]` and `ExactAlgebraicQ[Root[##^3 - ## + parameter &, 1]]` are `True` (C01, confirmed; `AlgebraicNumber[Pi, {1, 1}]` was already rejected because its first argument is not exact algebraic in the grammar). With `numericallyDifferentQ` forced to `True`, `EqualityStatus[Sqrt[5 + 2 Sqrt[6]], Sqrt[2] + Sqrt[3]]` returns `"Different"` (C02, confirmed). `powerProposals[(3 + 2 Sqrt[2])^(1/6), (1 + Sqrt[2])^(1/3)]` with no recursion and no trials returns the target, discarding the certified cheaper incumbent (C03, confirmed with R8's probe); `kummerCandidates[(3 + 2 Sqrt[2])^(1/4), 3 + 2 Sqrt[2], 1, 4]` with `recurse` replaced by the identity returns the target (C04, confirmed); the public calls still find $\sqrt{1+\sqrt{2}}$ and $(1+\sqrt{2})^{1/3}$, through the multiplier search, in 8.1 and 11.7 s. After a failed search with no budget the memo holds an entry for $\sqrt{1+\sqrt{2}}$ (C05, confirmed). `Strad[Sqrt[2], 17]` and `Strad[]` return unevaluated, and `DenestReport[1, "TimeBudget" -> 0]` reports a computed `InitialCost` (C06, C08, confirmed). `RadicalCost[1 + 2^100 I]` equals `RadicalCost[1 + I]` (C09, confirmed). With `"MaxTrials" -> 0` the three-surd examples $\sqrt{118+2\sqrt{210}+14\sqrt{55}+2\sqrt{462}}$ and $\sqrt{37+4\sqrt{15}+6\sqrt{14}+2\sqrt{210}}$ are returned unchanged, so the fast path misses them; with the default budget the multiplier search finds them in 2.2 and 0.4 s (C12, confirmed). The fifth, seventh and ninth roots and Ramanujan's $\sqrt{28^{1/3}-3}$ are already denested by `StradFixed2.wl` in 0.02–0.16 s through the Kummer linear factors and the multiplier search, so C13 and C14 are fast paths for identities the search already covered, not coverage gaps in the public function.

3 The consolidated catalogue

Table 2 lists every issue addressed by `StradFixed3.wl`: the reviews' findings merged where they describe the same defect, and the observations made during this session (marked "this session"). "Disposition" says what the new version does; the section references point to the design description.

Table 2: Consolidated catalogue of the issues addressed by `StradFixed3.wl`.

ID	Issue	Sources	Disposition
C01	`ExactAlgebraicQ` admits every exact `Root` and `AlgebraicNumber`, including `Root[##^5 + ## - Pi &, 1]` and parametric roots.	R7 F01, R8 F01, R9 F1	Fixed. `rootPolynomial` validates the defining polynomial (single or triangular-system form), its coefficients, degree and index; `AlgebraicNumber` needs an admitted generator and Gaussian-rational coefficients (§4.1).
C02	The numeric prefilter decides the public status `"Different"` and is counted as a certificate.	R7 F02, R8 F05, R9 F7	Fixed. `certify` is exact in every branch; any nonzero exact algebraic result of `RootReduce` proves inequality; the prefilter is search-layer pruning in `accept`, off by default, counted separately (§4.2).
C03	`negativeRadicandCandidate` and `kummerCandidates` start from the target and overwrite a cheaper incumbent.	R7 F03, R8 F02, R9 F2	Fixed. Both take the incumbent as a fifth argument and return it unchanged when they find nothing (§4.3).
C04	The reduced presentation $\gamma^{1/(q/k)}$ is offered only when recursion changes it and lowers its depth.	R7 F04, R8 F03, R9 F3	Fixed. The raw reduced presentation and its orbit are offered first, recursion afterwards; the same for the even-index split; the values $\gamma\zeta_k^j$ are processed once each (§4.3).
C05	Unchanged results are memoized without the budget of the search; a quarter-budget recursive failure answers a full-budget visit.	R7 F06, R8 F04, R9 F4	Adopted differently. Memo entries carry result, budget and completeness; a negative entry is reused only by a search with no more budget; `$inProgress` cuts cycles (§4.4).
C06	`FactorInteger`, `Together`, `Expand` and the fast-path dispatcher run unwrapped; input classification and report costs run outside the region.	R7 F05, R8 F06, R9 F5	Fixed. All wrapped in `bounded[]`; classification and both costs inside the region; `Missing["NotComputed"]` when disabled or interrupted (§4.5).
C07	The discriminant batch limit of 24 is checked in the outer loop; one prime can raise a batch to 46.	R9 F6, R8 F12	Fixed. Checked in the inner loop; option `"DiscriminantBatchCap"` (§4.5).
C08	`Strad[e, 17]` and `Strad[]` stay unevaluated instead of returning a `Failure`.	R7 F07, R8 F14, R9 F8	Fixed. Catch-all argument sequences reach the validator; the no-argument call is a `Failure` (§4.6).

ID	Issue	Sources	Disposition
C09	Cost: Gaussian components are not priced; the opaque node count is subtractive.	R7 F11, R9 F9	Fixed. Recursive `bitSize`, `radicalNodes` and `opaqueCount` that stop at opaque objects (§4.7).
C10	`FastPathAccepted` counts proposals, not acceptances.	R7 F08	Fixed. Compared before and after the fast-path stage.
C11	Certificate records are a truncated sample of accepted proposals but read as a proof chain; the result-changed fact is folded into the status.	R7 F09, R8 F13	Fixed. `CertificateKind`, `CertificatesTruncated`, `ResultChanged`; each record names its `EqualityMethod` and `Scope` (§4.10).
C12	The multi-surd unknown sets miss the cosets of the square-class group of the radicands ($\sqrt{6}+\sqrt{35}+\sqrt{77}$ and three more examples).	R7 F10, R8 F07, R9 F10	Fixed. Coset enumeration over the primes of radicands and coefficients, an integer-relation stage per coset, rational systems as fallback; `"MaxCosets"` (§4.8, §5.2).
C13	Honsbeek recognizer requires two terms of the form $c\,r^{1/3}$; rational summands and terms like $2^{2/3}7^{1/3}$ are not recognized.	R8 F08; this session	Fixed. Any depth-one term with radical indices dividing 3 whose cube reduces to a nonzero rational; both signs; negative denominators give complex candidates for the gate (§4.9).
C14	The trace–norm fast path stops at index 3.	R8 F09, R7 §6, R9 §8.6	Fixed. `quadraticOddRoots` for every odd $q \le$ `"MaxOddIndex"` (default 9) by the Dickson recurrences (§5.1).
C15	Syntactic multiplier keys count equal multipliers twice.	R8 F12	Not adopted; documented. Canonicalizing every proposal was the dominant cost removed in unified-B.
C16	Independent list progress; exact islands inside inexact hosts.	R8 F10, F11	Not adopted; documented policies (§7).
C17	Character reconstruction over $\mathbb{F}_2^r$ as the multi-surd repair.	R9 §9	Adopted in the coset form (C12); the equivalence is Section 5.2.
C18	`Solve[..., Rationals]` may return conditional or parametric solutions; only fully rational solutions are candidates.	this session	Fixed in the new `surdSystem`; unified-B's version relied on the gate rejecting them.
C19	Multi-surd terms with rational radicands ($\sqrt{3/2}$, which the kernel writes for $\sqrt{6}/2$) are not parsed, so $\sqrt{(5+\sqrt{6}+\sqrt{10}+\sqrt{15})/2}$ misses the fast path.	this session	Fixed. `surdTerm` reads $c\sqrt{a/b}$ as $\sqrt{ab}$ with coefficient c/b (§4.8).
C20	An untagged `Throw` from a user solver escapes `Catch[..., _]` and aborts the call.	R7 (native test, first failing run); this session	Fixed. Nested plain and tagged `Catch`.
C21	`acceptWithPolish` polishes candidates that are already too large or after the deadline.	this session	Fixed. Size and expiry checks before the polish variants.

ID	Issue	Sources	Disposition
C22	In the first build of the coset search, a negative coefficient put -1 into the prime universe (`FactorInteger[-48]` starts with $\{-1, 1\}$), so the cosets contained negative radicands, the relation stage failed and the rational systems ran to their operation limit: 12–30 s per input in families F8, F10 and F11.	this session (randomized families)	Fixed. `classPrimes` factors absolute values and keeps primes only; the six slow inputs take 0.1–0.3 s; two regression tests (negative coefficients; relation stage suffices).
C23	The patience rule of the multiplier search counted trials only since an improvement found *inside* the search, so an island already improved by index reduction (C04) ran all 120 trials: $(3 + 2\sqrt{2})^{1/6}$ with `"MaxRecursion" -> 0` took 31 s against 1.7 s in unified-B.	this session (battery timing)	Fixed. Patience counts from the last improvement whatever stage produced it; one regression test.

4 The design changes of `StradFixed3.wl`

The package lives in the context `RadicalDenest3‘`; it can be loaded next to `RadicalDenest2‘` and `RadicalDenest‘` for differential testing, which Section 6.8 does. The public interface and the option names of `StradFixed2.wl` are unchanged; three options are added and one default changes:

Option	Default	Meaning
`"MaxOddIndex"`	9	largest odd index handled by the trace–norm recipe in a real quadratic field
`"DiscriminantBatchCap"`	24	proposals generated from one discriminant, checked in the inner loop
`"MaxCosets"`	16	cosets tried by the multi-surd search
`"NumericPrefilter"`	`False` (was `True`)	optional numeric pruning of search candidates; never decides a status

The complete source (939 lines) is reproduced in Appendix A. The following subsections describe what changed relative to unified-B.

4.1 Grammar

`algebraicFormQ` still admits rationals, Gaussian rationals and their `Plus`, `Times` and rational-`Power` combinations. A `Root` object is admitted only through `rootPolynomial`: the head must be `Root` with two or three arguments; the third, if present, is 0 or 1; the first argument is either a pure function whose value at a fresh variable expands to a nonconstant polynomial with coefficients in the radical grammar and whose index is between 1 and the degree, or a list of pure functions with a list of indices of the same length, the triangular-system form the kernel uses for a root of a polynomial with algebraic coefficients (`Root[##^3 + Sqrt[2] ## + 1 &, 1]` evaluates to `Root[{-2 + ##1^2 &, 1 + ##1 ##2 + ##2^3 &}, {2, 1}]`), each polynomial in the system having coefficients in the radical grammar. An `AlgebraicNumber` needs an admitted first argument and a list of Gaussian rationals. `Root[##^5 + ## - Pi &, 1]`, `Root[##^3 - ## + a &, 1]` and `AlgebraicNumber[Pi, {1, 1}]` are rejected; a rejected object inside a host is an opaque host node, so `Strad[Root[##^5 + ## - Pi &, 1] + Sqrt[5 + 2 Sqrt[6]]]` still denests the radical and leaves the root alone (tested).

4.2 Certification and the gate

certify[a, b] returns "Unknown" unless both arguments are admitted; "Equal" if they are identical or RootReduce[a - b] is 0 or PossibleZeroQ[a - b, Method -> "ExactAlgebraics"] is True; "Different" if the reduced difference is a nonzero exact algebraic number (canonicalNonzeroQ now accepts any admitted nonzero expression free of an unevaluated RootReduce, so $2\sqrt{2}$ proves $\sqrt{2} \neq -\sqrt{2}$) or PossibleZeroQ answers False; and "Unknown" otherwise. Every step is bounded. The method that decided is recorded in $lastCertificateMethod and copied into the certificate record. numericallyDifferentQ is called only by accept, before certify, and only when "NumericPrefilter" is True; its rejections are counted in NumericRejections and no longer in CertificatesDifferent. acceptWithPolish returns immediately after the deadline or for oversized candidates.

4.3 Proposal stages and incumbents

powerProposals passes its incumbent to negativeRadicandCandidate[target, rho, p, q, incumbent] and kummerCandidates[target, rho, p, q, incumbent]; both default the fifth argument to the target for standalone use and return the incumbent when they find nothing. kummerCandidates collects the distinct values $\gamma\zeta_k^j$ over all linear factors γ and all j first, then for each such δ offers $(\delta^{1/(q/k)}\zeta_{q/k}^l)^p$ for every l through the gate before it recurses into $\delta^{1/(q/k)}$; the even-index split likewise offers $(\sqrt{\rho})^{2/q}$ before recursing. Since the gate compares costs, $\sqrt{1+\sqrt{2}}$ replaces $(3+2\sqrt{2})^{1/4}$ and $(1+\sqrt{2})^{1/3}$ replaces $(3+2\sqrt{2})^{1/6}$ with "MaxRecursion" -> 0 (tested). FastPathAccepted is incremented only when the fast-path stage changed the incumbent.

4.4 The memo

improveNumber stores for every island ⟨result, budget, complete⟩ with budget = {"MaxTrials" in force, remaining recursion levels} and complete = not expired. On a later visit a stored improvement becomes the incumbent; the visit returns it without searching only if the stored search was complete and had at least the current budget componentwise. Islands on the current recursion path ($inProgress) are not re-entered. The reviews' tests translate to: a failed search at recursion level 1 followed by two visits at level 0 runs the multiplier search twice, not three times; an in-progress island returns its incumbent (tested).

4.5 Budgets

FactorInteger (discriminants, square classes), Together in linearRoots, the Expand calls of the ansatz and the fast-path dispatcher run inside bounded[]. In run, the classification of the input (inexact, not algebraic) and the initial cost are computed inside the TimeConstrained/MemoryConstrained region; the committed pass carries its cost, so no cost is computed after an interruption; a disabled or interrupted call reports Missing["NotComputed"] where no cost was computed. The discriminant batch counter is checked before every proposal, in the inner loop over exponents as well as the outer loop over primes, against "DiscriminantBatchCap".

4.6 Dispatch

Each public head has two definitions, head[e_, all:(True|False), args___] and head[e_, args___], both forwarding {args} to resolveOptions, which flattens nested lists and requires every element to be a rule; head[] returns Failure["InvalidArguments", ...]. Strad[e, 17], Strad[], DenestReport[1, 2, 3] are all Failures (tested).

4.7 Cost

`opaqueCount`, `radicalNodes` and `bitSize` are recursive traversals that stop at `Root` and `AlgebraicNumber` objects; `bitSize` prices a `Complex` as the sum of its parts, so $1 + 2^{100}i$ is costlier than $1 + i$. The five-tuple `RadicalCost` is otherwise unchanged.

4.8 The coset search

`multiSurdSquareRoots` expands the radicand, parses every summand with `surdTerm` into a squarefree radicand and a rational coefficient (reading $c\sqrt{a/b}$ as $\sqrt{ab}$ with coefficient c/b), and needs at least two and at most fifteen radicands. `cosetBases[radicands, primes]` enumerates the cosets of the group H generated by the radicands' square classes over the primes of the radicands and of the coefficients (`classPrimes`), at most `"MaxCosets"` of them, each as a sorted list of squarefree integers. Stage one runs `surdRelation` on every coset U: `FindIntegerNullVector` on $(\sqrt{|\rho|}, \sqrt{u_1}, \ldots)$ at $80 + 10|U|$ digits gives an integer vector (n_0, n_u); if $n_0 \neq 0$ the candidate $-\sum n_u\sqrt{u}/n_0$ (times i for negative ρ) is kept only when `RootReduce` of its square minus ρ is exactly 0, so a spurious relation costs one bounded `RootReduce`. Stage two, reached only if stage one finds nothing, solves the rational systems of unified-B for the two classical unknown sets and then for the cosets (`surdSystem`), keeping only solutions that assign a rational to every unknown (C18). R8's names `cosetBases` and `surdSystem` were adopted so that its tests translate literally.

4.9 Honsbeek and odd indices

`honsbeekSquareRoots` accepts a two-term sum whose terms have depth at most one, are real, have radical indices dividing 3, and whose cubes reduce to nonzero rationals a, b (a rational summand is its own cube root; at least one term must be irrational); it offers $\pm N/\sqrt{b - s^3 a}$ for every rational root s of $x^4+4x^3+8(b/a)x-4b/a$ with $b-s^3a \neq 0$, the gate selecting the branch. `quadraticOddRoots[{a, b, c}, q]` implements Section 5.1 for odd $3 \leq q \leq$ `"MaxOddIndex"`; `cubicInQuadratic` is now its $q = 3$ instance, and `rootCandidates` dispatches every odd index in range to it.

4.10 Reporting

`DenestReport` adds `ResultChanged`, `CertificateKind` (a sentence stating that the records are kernel-checked accepted proposals, a bounded sample and not a proof chain), and `CertificatesTruncated` (`CandidatesAccepted` exceeds the number of records); every record carries `EqualityMethod` (`"SameQ"`, `"RootReduce"` or `"PossibleZeroQ/ExactAlgebraics"`) and `Scope -> "AcceptedProposal"`. `Statistics` gains `CosetSystems`.

5 The mathematics of the coverage extensions

5.1 Odd roots in a real quadratic field

Let $K = \mathbb{Q}(\sqrt{c})$ with $c > 1$ squarefree, $\rho = a + b\sqrt{c} \in K$ with $b \neq 0$, $q \geq 3$ odd. Write $\bar{\rho} = a - b\sqrt{c}$, $\mathrm{N}\,\rho = \rho\bar{\rho} = a^2 - b^2c$. For $\beta = A + B\sqrt{c} \in K$ let $T = \mathrm{Tr}\,\beta = 2A$ and $n = \mathrm{N}\,\beta = A^2 - B^2c$. The Dickson polynomials $D_q(T, n)$ and $S_q(T, n)$ are defined by $D_q(u + v, uv) = u^q + v^q$ and $S_q(u + v, uv) = (u^q - v^q)/(u - v)$, equivalently by $D_0 = 2$, $D_1 = T$, $D_{k+1} = TD_k - nD_{k-1}$ and $S_0 = 0$, $S_1 = 1$, $S_{k+1} = TS_k - nS_{k-1}$.

Proposition 5.1. *The real q-th root $\rho^{1/q}$ lies in K if and only if $n := \mathrm{sgn}(\mathrm{N}\,\rho)\,|\mathrm{N}\,\rho|^{1/q}$ is rational and the polynomial $D_q(T, n) - 2a$ has a rational root T. In that case $S_q(T, n) \neq 0$ and*

$$\rho^{1/q} = \frac{T}{2} + \frac{b\sqrt{c}}{S_q(T, n)}.$$

Proof. If $\beta = A + B\sqrt{c} \in K$ satisfies $\beta^q = \rho$, then $\mathrm{N}\,\beta^q = \mathrm{N}\,\rho$, so $n = \mathrm{N}\,\beta$ is the real q-th root of $\mathrm{N}\,\rho$ (odd q, sign preserved) and is rational; and with $u = \beta$, $v = \bar{\beta}$, $u + v = T$, $uv = n$, we get $D_q(T, n) = \beta^q + \bar{\beta}^q = \rho + \bar{\rho} = 2a$, so T is a rational root. Conversely, given rational n and T with $T^2 - 4n \geq 0$ or not, let u, v be the roots of $X^2 - TX + n$; then $u^q + v^q = 2a$ and $u^q v^q = n^q = \mathrm{N}\,\rho$, so $\{u^q, v^q\} = \{\rho, \bar{\rho}\}$; choosing $u^q = \rho$ gives $u \in \mathbb{R}$ with $u^q = \rho$, hence $u = \rho^{1/q}$, and $u - v = (u^q - v^q)/S_q(T, n) = 2b\sqrt{c}/S_q(T, n)$ provided $S_q(T, n) \neq 0$. If $S_q(T, n) = 0$ then $u^q = v^q$, so $\rho = \bar{\rho}$ and $b = 0$, excluded. Hence $u = (T + (u - v))/2$, the displayed formula, and $u \in K$. □

The implementation computes D_q and S_q by the recurrences, finds the rational roots of $D_q(x, n) - 2a$ with `rationalRoots`, and certifies each candidate by `RootReduce` of $\beta^q - \rho$ before offering it. For $q = 3$, $D_3 = T^3 - 3nT$ and $S_3 = T^2 - n$ recover the cubic recipe of unified-B. A negative radicand is handled by the caller: $\rho < 0$ gives $(-1)^{1/q}(-\rho)^{1/q}$.

5.2 Square roots of multi-surd sums: the single-coset theorem

Let $\rho = \sum_{d \in D} a_d \sqrt{d}$ with $a_d \in \mathbb{Q}$, D a finite set of squarefree positive integers including 1, and let $H \leq \mathbb{Q}^\times/\mathbb{Q}^{\times 2}$ be the group generated by the classes of D; identify each class with its squarefree representative. Let $L = \mathbb{Q}(\sqrt{h} : h \in H)$, a multiquadratic field of degree $|H|$ with $\mathbb{Q}$-basis $\{\sqrt{h} : h \in H\}$.

Theorem 5.2 (single-coset support)**.** *Suppose $\sqrt{\rho} = \sum_{u \in U} x_u \sqrt{u}$ with $x_u \in \mathbb{Q}^\times$ and U a set of squarefree positive integers. Then U is contained in a single coset dH of H in $\mathbb{Q}^\times/\mathbb{Q}^{\times 2}$.*

Proof. Let G be the group generated by H and the classes of U, $M = \mathbb{Q}(\sqrt{g} : g \in G) \supseteq L$, with basis $\{\sqrt{g} : g \in G\}$; $\mathrm{Gal}(M/\mathbb{Q}) \cong \mathrm{Hom}(G, \{\pm 1\})$ acts by $\sigma_\chi(\sqrt{g}) = \chi(g)\sqrt{g}$. Take χ trivial on H, so σ_χ fixes $L \ni \rho$. Then $\sigma_\chi(\sqrt{\rho})^2 = \rho$, so $\sigma_\chi(\sqrt{\rho}) = \pm\sqrt{\rho}$, i.e. $\sum \chi(u) x_u \sqrt{u} = \pm \sum x_u \sqrt{u}$; by linear independence of the $\sqrt{u}$, $\chi(u) = \epsilon_\chi$ is the same sign for all $u \in U$. Hence for $u, u' \in U$, $\chi(u/u') = 1$ for every χ trivial on H, so $u/u' \in H$ (characters of the finite group G/H separate points). □

Corollary 5.3. *If $\sqrt{\rho}$ is a rational combination of square roots of positive integers, then it is a rational combination of $\{\sqrt{u} : u \in dH\}$ for one coset dH, and d can be taken as a product of primes dividing some element of D or the numerator or denominator of some a_d.*

Proof. The first statement is the theorem. For the second, $\sqrt{\rho} \in M = L(\sqrt{d})$ and $\sqrt{\rho}^2 = \rho \in L$; the field $L(\sqrt{\rho})$ is unramified outside the primes dividing 2, the elements of D and the numerators and denominators of the a_d (the discriminant of $X^2 - \rho$ over L is 4ρ), and $L(\sqrt{\rho}) = L(\sqrt{d})$ forces the class d to be supported on those primes. □

This is the gap the reviews proved: for $\rho = 118 + 2\sqrt{210} + 14\sqrt{55} + 2\sqrt{462}$, $H = \{1, 55, 210, 462\}$ and $\sqrt{\rho} = \sqrt{6} + \sqrt{35} + \sqrt{77}$ has support in the coset $6H = \{6, 35, 77, 330\}$, which neither $\{1, 2, 3, 5, 7, 11\}$ nor $\{1, 2, 3, 5, 7, 11, 55, 210, 462\}$ contains. `cosetBases` enumerates the cosets dH for d ranging over the classes generated by the prime set, so by the corollary a rational-combination square root, if one exists, is a rational combination of one of the enumerated bases (subject to the cap `"MaxCosets"`). Within a coset, the coefficient vector is a solution of a system of $|H|$ quadratic equations in $|H|$ unknowns (stage two), or, more cheaply, an integer relation between $\sqrt{\rho}$ and the basis (stage one): if $\sqrt{\rho} = \sum x_u \sqrt{u}$ with $x_u = p_u/s$, then $(s, -p_{u_1}, \ldots)$ is an integer null vector of $(\sqrt{\rho}, \sqrt{u_1}, \ldots)$, and `FindIntegerNullVector` finds a null vector of small height when one exists at the working precision. The numeric step is a proposal only: the candidate is offered after `RootReduce` has verified its square, and the gate certifies it against the island. R9's character reconstruction computes the same coefficients as $F_g = \frac{1}{|H|} \sum_\sigma \chi_g(\sigma) \epsilon_\sigma \sqrt{\sigma\rho}$ over the $|H|$ conjugates and $2^{|H|-1}$ sign patterns; the two methods search the same space, and the relation stage is the cheaper of the two.

5.3 Honsbeek with a rational summand

Honsbeek's identity: for α, β with $\alpha^3 = a$, $\beta^3 = b$ rational nonzero and s a rational root of $x^4 + 4x^3 + 8(b/a)x - 4b/a$ with $b - s^3 a \neq 0$,

$$\sqrt{\alpha + \beta} = \pm \frac{-\frac{s^2}{2}\alpha^2 + s\alpha\beta + \beta^2}{\sqrt{b - s^3 a}}.$$

Unified-B proved it for two irrational cube roots; the proof is an identity in α, β subject to $\alpha^3 = a$, $\beta^3 = b$ and the quartic, and does not use irrationality, so a rational summand $\beta = -3$ (that is $b = -27$) is admissible. With $\alpha = 28^{1/3}$, $\beta = -3$, $b/a = -27/28$, the quartic is $x^4 + 4x^3 - \frac{54}{7}x + \frac{27}{7} = \frac{1}{7}(x+3)(7x^3 + 7x^2 - 21x + 9)$, with the single rational root $s = -3$; then $b - s^3 a = -27 + 27 \cdot 28 = 729 = 27^2$ and the numerator is $-\frac{9}{2}\,28^{2/3} + 9 \cdot 28^{1/3} + 9$, so the candidate is $\mp\frac{1}{3}(98^{1/3} - 28^{1/3} - 1)$ (using $28^{2/3} = 2 \cdot 98^{1/3}$), and the gate keeps the positive sign: Ramanujan's $\sqrt{28^{1/3} - 3} = \frac{1}{3}(98^{1/3} - 28^{1/3} - 1)$, recovered in 0.07 s (log `logs/exp_battery3.txt`, row N16). The term $4^{1/3}$ of the classical $\sqrt{5^{1/3} - 4^{1/3}}$ is written $2^{2/3}$ by the kernel; the new recognizer accepts it because its cube reduces to 4. When $b - s^3 a < 0$ the denominator is imaginary and the candidates are complex; they are offered and the gate discards them for a real target.

6 Experiments

6.1 Setup

All runs use the scripts in `harness/`, which load `src/corrected/StradFixed3.wl` (or, for the side-by-side columns, `StradFixed2.wl` through unified-B's `runner2.wl`) and record every result as unified-A did: `runCase` captures the value, the messages, the wall time, exact equality with the expected value by `RootReduce`, and the radical depth before and after. The transcripts are in `logs/`; the tables are produced by `harness/export_tables3.wl` from the recorded rows. One kernel ran at a time; `harness/run_all.sh` is the chain.

6.2 The battery

Tables 3–7 repeat the battery of unified-A/B on `StradFixed3.wl`, next to the depth and time recorded in unified-B for `StradFixed2.wl`. Every output is exactly equal to its input and no kernel message was emitted. Of the 116 rows, 71 reduce the radical depth, the same 71 as before, and every result is syntactically identical to the result of `StradFixed2.wl` (except the private marker symbol of row E09, which carries the new context). The total wall time fell from 108.8 s to 59.7 s; the rows that took more than five seconds before (A27, A30, K04, L09) take 2.4, 6.2, 0.6 and 26.2 s now. The one row that became slower is L09, $(5 + 2\sqrt{6})^{1/6}$, which is not denestable: the index-reduction stage now searches the sub-islands $\pm(5 + 2\sqrt{6})^{1/3}$ and $\pm(5 + 2\sqrt{6})^{1/2}$ before the multiplier search runs on the target (C04), and the budget-aware memo does not let a recursive failure shorten the top-level search (C05).

Table 3: Classical identities: second corrected version (unified-B) against `StradFixed3`. Depth is the syntactic radical depth before and after; a $\times$ after a depth would mark an output not exactly equal to its input.

ID	Input	StradFixed2		StradFixed3		Output of StradFixed3
		depth	time (s)	depth	time (s)	
A01	`Strad[Sqrt[3 + 2*Sqrt[2]]]`	2→1	0.438	2→1	0.030	`1 + Sqrt[2]`
A02	`Strad[Sqrt[5 + 2*Sqrt[6]]]`	2→1	0.167	2→1	0.024	`Sqrt[2] + Sqrt[3]`
A03	`Strad[Sqrt[2 + Sqrt[3]]]`	2→1	0.205	2→1	0.027	`(1 + Sqrt[3])/Sqrt[2]`
A04	`Strad[Sqrt[11 + 6*Sqrt[2]]]`	2→1	0.144	2→1	0.011	`3 + Sqrt[2]`
A05	`Strad[Sqrt[3 + Sqrt[5]]]`	2→1	0.391	2→1	0.026	`(1 + Sqrt[5])/Sqrt[2]`
A06	`Strad[Sqrt[7 + 2*Sqrt[10]]]`	2→1	0.132	2→1	0.015	`Sqrt[2] + Sqrt[5]`
A07	`Strad[Sqrt[6 + 2*Sqrt[2] + 2*Sqrt[3] + 2*Sqrt[6]]]`	2→1	0.716	2→1	0.033	`1 + Sqrt[2] + Sqrt[3]`
A08	`Strad[Sqrt[12 + 2*Sqrt[6] + 2*Sqrt[14] + 2*Sqrt[21]]]`	2→1	0.290	2→1	0.060	`Sqrt[2] + Sqrt[3] + Sqrt[7]`
A09	`Strad[(Sqrt[5] - 2)^(1/3)]`	2→1	0.098	2→1	0.022	`(-1 + Sqrt[5])/2`
A10	`Strad[(2 + Sqrt[5])^(1/3)]`	2→1	0.083	2→1	0.019	`(1 + Sqrt[5])/2`
A11	`Strad[(10 + 6*Sqrt[3])^(1/3)]`	2→1	0.068	2→1	0.011	`1 + Sqrt[3]`
A12	`Strad[(2^(1/3) - 1)^(1/3)]`	2→1	1.200	2→1	0.369	`(1 - 2^(1/3) + 2^(2/3))/3^(2/3)`
A13	`Strad[(5^(1/3) - 4^(1/3))^(1/3)]`	2→2	4.505	2→2	1.255	`(-2^(2/3) + 5^(1/3))^(1/3)`
A14	`Strad[((11 + 5*Sqrt[5])/2)^(1/5)]`	2→1	0.121	2→1	0.021	`(1 + Sqrt[5])/2`
A15	`Strad[Sqrt[1 + 2*2^(1/3) + 2^(2/3)]]`	2→1	0.072	2→1	0.019	`1 + 2^(1/3)`
A16	`Strad[(17 + 12*Sqrt[2])^(1/4)]`	2→1	0.060	2→1	0.029	`1 + Sqrt[2]`
A17	`Strad[(99 + 70*Sqrt[2])^(1/6)]`	2→1	0.127	2→1	0.042	`1 + Sqrt[2]`
A18	`Strad[(3 + 2*Sqrt[2])^(3/2)]`	2→1	0.039	2→1	0.015	`7 + 5*Sqrt[2]`
A19	`Strad[(3 + 2*Sqrt[2])^(-2^(-1))]`	2→1	0.058	2→1	0.024	`-1 + Sqrt[2]`
A20	`Strad[1/Sqrt[3 + 2*Sqrt[2]]]`	2→1	0.061	2→1	0.020	`-1 + Sqrt[2]`
A21	`Strad[Sqrt[3 + 2*Sqrt[2]] + Sqrt[5 + 2*Sqrt[6]]]`	2→1	0.128	2→1	0.035	`1 + 2*Sqrt[2] + Sqrt[3]`
A22	`Strad[Sqrt[3 + 2*Sqrt[2]] *Sqrt[5 + 2*Sqrt[6]]]`	2→1	0.264	2→1	0.040	`2 + Sqrt[2] + Sqrt[3] + Sqrt[6]`
A23	`Strad[Sqrt[2 + Sqrt[3]] + Sqrt[2 - Sqrt[3]]]`	2→1	0.221	2→1	0.077	`Sqrt[6]`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
A24	`Strad[Sqrt[-1 + 2*I*Sqrt[2]]]`	2→1	0.155	2→1	0.064	`1 + I*Sqrt[2]`
A25	`Strad[Sqrt[2 + 2*I*Sqrt[3]]]`	2→1	0.049	2→1	0.021	`I + Sqrt[3]`
A26	`Strad[Sqrt[-3 - 2*Sqrt[2]]]`	2→1	0.051	2→1	0.020	`I*(1 + Sqrt[2])`
A27	`Strad[(3 + 2*Sqrt[2])^(1/4)]`	2→2	16.650	2→2	2.436	`Sqrt[1 + Sqrt[2]]`
A28	`Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]], True]`	3→2	1.492	3→2	0.613	`Sqrt[5 + 2*Sqrt[2]]`
A29	`Strad[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]`	3→2	2.151	3→2	0.344	`Sqrt[5 + 2*Sqrt[2]]`
A30	`Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]], True]`	3→2	11.966	3→2	6.167	`Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]`
A31	`Strad[Sqrt[16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]]]]`	3→2	4.555	3→2	2.390	`Sqrt[5] + Sqrt[11 - 2*Sqrt[29]]`

Table 4: Non-denestable inputs (B) and branch counterexamples (C).

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
B01	`Strad[Sqrt[2 + Sqrt[2]]]`	2→2	0.960	2→2	0.409	`Sqrt[2 + Sqrt[2]]`
B02	`Strad[Sqrt[1 + Sqrt[2]]]`	2→2	1.016	2→2	0.434	`Sqrt[1 + Sqrt[2]]`
B03	`Strad[(1 + Sqrt[2])^(1/3)]`	2→2	1.349	2→2	0.509	`(1 + Sqrt[2])^(1/3)`
B04	`Strad[Sqrt[2]]`	1→1	0.001	1→1	0.001	`Sqrt[2]`
B05	`Strad[2]`	0→0	0.002	0→0	0.000	`2`
B06	`Strad[Sqrt[1 + I]]`	1→1	0.002	1→1	0.000	`Sqrt[1 + I]`
B07	`Strad[Root[#1^4 - 10*#1^2 + 1 & , 4]]`	0→1	0.081	0→1	0.028	`Sqrt[2] + Sqrt[3]`
B08	`Strad[Sqrt[Sqrt[2] + Sqrt[3]]]`	2→2	1.666	2→2	0.591	`Sqrt[Sqrt[2] + Sqrt[3]]`
B09	`Strad[(2 + 2*I*Sqrt[3])^(1/3)]`	2→2	3.043	2→2	0.931	`(2 + (2*I)*Sqrt[3])^(1/3)`
C01	`Strad[-Sqrt[3 + 2*Sqrt[2]]]`	2→1	0.051	2→1	0.013	`-1 - Sqrt[2]`
C02	`Strad[(-1 + 2*I*Sqrt[2])^(3/2)]`	2→1	0.091	2→1	0.027	`-5 + I*Sqrt[2]`
C03	`Strad[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]]]`	2→1	0.225	2→1	0.067	`-((-I + Sqrt[2])*(-I + Sqrt[3]))`
C04	`Strad[(-3 - 2*Sqrt[2])^(3/2)]`	2→1	0.093	2→1	0.025	`-7*I - (5*I)*Sqrt[2]`
C05	`Strad[Sqrt[-2 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]`	2→2	3.448	2→2	0.486	`-Sqrt[2*(7 + 5*Sqrt[2])]`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
C06	`Strad[Sqrt[-3 - 2*Sqrt[2]]*Sqrt[-3 - 2*Sqrt[2]]]`	1→1	0.002	1→1	0.001	`-3 - 2*Sqrt[2]`
C07	`Strad[(-1 + 2*I*Sqrt[2])^(5/2)]`	2→1	0.069	2→1	0.023	`1 - (11*I)*Sqrt[2]`
C08	`Strad[(-5 - 2*Sqrt[6])^(3/2)]`	2→1	0.088	2→1	0.035	`(-11*I)*Sqrt[2] - (9*I)*Sqrt[3]`

Table 5: Direct core calls with forced multipliers (D) and symbolic input (E).

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
D01	`DenestCore[-Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {1}]`	2→1	0.030	2→1	0.012	`-1 - Sqrt[2]`
D02	`DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {0}]`	2→1	0.034	2→1	0.010	`1 + Sqrt[2]`
D03	`DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {}]`	2→1	0.024	2→1	0.010	`1 + Sqrt[2]`
D04	`DenestCore[(-1 + 2*I*Sqrt[2])^(3/2), "Multipliers" -> {1}]`	2→1	0.030	2→1	0.019	`-5 + I*Sqrt[2]`
D05	`DenestCore[Sqrt[-1 + 2*I*Sqrt[2]]*Sqrt[-2 + 2*I*Sqrt[3]], "Multipliers" -> {1}]`	2→1	0.224	2→1	0.180	`-((-I + Sqrt[2])*(-I + Sqrt[3]))`
D06	`DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, 6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]`	2→1	0.020	2→1	0.013	`Sqrt[2] + Sqrt[3]`
D07	`DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {6 - 2*Sqrt[6] - 2*Sqrt[2] + 2*Sqrt[3]}]`	2→1	0.017	2→1	0.012	`Sqrt[2] + Sqrt[3]`
D08	`DenestCore[Sqrt[3 + 2*Sqrt[3 + 2*Sqrt[2]]]]`	3→2	1.341	3→2	0.153	`Sqrt[5 + 2*Sqrt[2]]`
D09	`DenestCore[2 + Sqrt[3]]`	1→1	0.001	1→1	0.000	`2 + Sqrt[3]`
D10	`DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {2.}]`	2→1	0.014	2→1	0.009	`1 + Sqrt[2]`
D11	`DenestCore[Sqrt[3 + 2*Sqrt[2]], "Multipliers" -> {Pi}]`	2→1	0.018	2→1	0.010	`1 + Sqrt[2]`
D12	`DenestCore[Sqrt[5 + 2*Sqrt[6]], "Multipliers" -> {1, Sqrt[5]}]`	2→1	0.024	2→1	0.013	`Sqrt[2] + Sqrt[3]`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
D13	`DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {-1}]`	2→1	0.022	2→1	0.011	`1 + Sqrt[2]`
D14	`DenestCore[Sqrt[3 + 2* Sqrt[2]], "Multipliers" -> {I}]`	2→1	0.018	2→1	0.013	`1 + Sqrt[2]`
D15	`DenestCore[Sqrt[5 + 2* Sqrt[6]], "Multipliers" -> {1}]`	2→1	0.026	2→1	0.015	`Sqrt[2] + Sqrt[3]`
E01	`Strad[Sqrt[z^2]]`	1→1	0.001	1→1	0.001	`Sqrt[z^2]`
E02	`Strad[Sqrt[t + t^2]]`	1→1	0.001	1→1	0.000	`Sqrt[t + t^2]`
E03	`Strad[Sqrt[-x - y]]`	1→1	0.001	1→1	0.001	`Sqrt[-x - y]`
E04	`Strad[Sqrt[a + b]]`	1→1	0.001	1→1	0.000	`Sqrt[a + b]`
E05	`Strad[x]`	0→0	0.001	0→0	0.000	`x`
E06	`Strad[Sqrt[x]*Sqrt[3 + 2* Sqrt[2]]]`	2→1	0.013	2→1	0.007	`(1 + Sqrt[2])*Sqrt[x]`
E07	`Strad[f[1]]`	0→0	0.001	0→0	0.000	`f[1]`
E08	`Strad[f[Sqrt[3 + 2*Sqrt[ 2]]]]`	2→2	0.001	2→2	0.000	`f[Sqrt[3 + 2*Sqrt[2]]]`
E09	`Strad[RadicalDenest3‘ Private‘mark[1]]`	0→0	0.001	0→0	0.000	`RadicalDenest3‘Private‘ mark[1]`

Table 6: Factorizer and rationalizer probes (J) and miscellaneous inputs (K).

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
J01	`Strad[Sqrt[-(Sqrt[2] + ( -1)^(1/3))]]`	2→2	1.670	2→2	0.946	`Sqrt[-(-1)^(1/3) - Sqrt[ 2]]`
J02	`Strad[Sqrt[-(Sqrt[2] + ( -1)^(1/3))] + 1]`	2→2	1.830	2→2	1.116	`1 + Sqrt[-(-1)^(1/3) - Sqrt[2]]`
J06	`Strad[(-(Sqrt[2] + (-1)^( 1/3)))^(1/3)]`	2→2	2.284	2→2	1.185	`(-(-1)^(1/3) - Sqrt[2])^ (1/3)`
J09	`Factorc[Sqrt[-(Sqrt[2] + (-1)^(1/3))]]`	2→2	0.001	2→2	0.001	`Sqrt[-(-1)^(1/3) - Sqrt[ 2]]`
J10	`Factorc[Sqrt[-3 - 2*Sqrt[ 2]]]`	2→2	0.001	2→2	0.000	`I*Sqrt[3 + 2*Sqrt[2]]`
J11	`Factorc[Sqrt[8 + 4*Sqrt[ 3]]]`	2→2	0.007	2→2	0.006	`2*Sqrt[2 + Sqrt[3]]`
J12	`Factorc[Sqrt[z^2]]`	1→1	0.000	1→1	0.000	`Sqrt[z^2]`
J13	`Factorc[Sqrt[-x - y]]`	1→1	0.000	1→1	0.000	`Sqrt[-x - y]`
J14	`Factorc[Sqrt[t + t^2]]`	1→1	0.000	1→1	0.000	`Sqrt[t + t^2]`
J15	`RationalizeDenominator[x/ (1 + Sqrt[2])]`	1→1	0.002	1→1	0.001	`-x + Sqrt[2]*x`
J16	`RationalizeDenominator[1/ 2 + 1/Sqrt[2]]`	1→1	0.000	1→1	0.000	`(1 + Sqrt[2])/2`
J17	`RationalizeDenominator[1/ x]`	0→0	0.000	0→0	0.000	`x^(-1)`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
K01	`Strad[Sqrt[28^(1/3) - 3]]`	2→1	0.204	2→1	0.076	`(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3`
K02	`Strad[(1 + 2^(1/3))^3]`	1→1	0.002	1→1	0.001	`(1 + 2^(1/3))^3`
K04	`Strad[Strad[Sqrt[16 - 2* Sqrt[29] + 2*Sqrt[55 - 10* Sqrt[29]]], True], True]`	2→2	9.961	2→2	4.907	`Sqrt[5] + Sqrt[11 - 2* Sqrt[29]]`
K09	`Strad[Sqrt[5 + 2*Sqrt[6]] /Sqrt[3 + 2*Sqrt[2]]]`	2→1	0.128	2→1	0.212	`2 - Sqrt[2] - Sqrt[3] + Sqrt[6]`
K10	`Strad[Exp[I*(Pi/7)]*Sqrt[ 3 + 2*Sqrt[2]]]`	2→1	0.017	2→1	0.010	`(1 + Sqrt[2])*E^((I/7)* Pi)`
K11	`Strad[Sqrt[3 + 2*Sqrt[2]] + Pi]`	2→1	0.019	2→1	0.009	`1 + Sqrt[2] + Pi`
K12	`Strad[Sqrt[3. + 2*Sqrt[2] ]]`	0→0	0.001	0→0	0.000	`2.414213562373095`
K14	`Strad[{Sqrt[3 + 2*Sqrt[2] ], Sqrt[5 + 2*Sqrt[6]]}]`	2→1	0.042	2→1	0.017	`{1 + Sqrt[2], Sqrt[2] + Sqrt[3]}`
K15	`Strad[Sqrt[3 + 2*Sqrt[2]] == 1 + Sqrt[2]]`	0→0	0.002	0→0	0.001	`True`
K17	`Strad[Sqrt[(3 + 2*Sqrt[2] )/(5 + 2*Sqrt[6])]]`	2→1	0.135	2→1	0.128	`-2 - Sqrt[2] + Sqrt[3] + Sqrt[6]`
K19	`Strad[Sqrt[Sqrt[2] + 1]^ 3]`	2→2	0.879	2→2	0.450	`(1 + Sqrt[2])^(3/2)`
K20	`Strad[Sqrt[2 + Sqrt[3]]* Sqrt[3 + 2*Sqrt[2]]]`	2→1	0.197	2→1	0.046	`((2 + Sqrt[2])*(1 + Sqrt[3]))/2`
K21	`Strad[Sqrt[3 + 2*Sqrt[2]] , "Verbose" -> False, "MaxTrials" -> 1]`	2→1	0.032	2→1	0.012	`1 + Sqrt[2]`
K22	`Strad[Sqrt[5 + 2*Sqrt[6]] , "Factor" -> False]`	2→1	0.038	2→1	0.015	`Sqrt[2] + Sqrt[3]`

Table 7: Higher-order roots (L) and roots of negative bases (Q).

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
L01	`Strad[(239 + 169*Sqrt[2]) ^(1/7)]`	2→1	0.066	2→1	0.012	`1 + Sqrt[2]`
L02	`Strad[(2 + Sqrt[2])^(1/5) ]`	2→2	1.349	2→2	0.658	`(2 + Sqrt[2])^(1/5)`
L03	`Strad[(1 + Sqrt[2])^(1/7) ]`	2→2	2.023	2→2	0.613	`(1 + Sqrt[2])^(1/7)`
L04	`Strad[Sqrt[17 + 2*Sqrt[6] + 2*Sqrt[10] + 2*Sqrt[14] + 2*Sqrt[15] + 2*Sqrt[21] + 2*Sqrt[35]]]`	2→1	0.344	2→1	0.539	`Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7]`
L05	`Strad[Expand[(1 + Sqrt[2] + Sqrt[3])^3]^(1/3)]`	2→1	0.135	2→1	0.037	`1 + Sqrt[2] + Sqrt[3]`
L06	`Strad[Expand[(Sqrt[2] + Sqrt[3])^4]^(1/4)]`	2→1	0.529	2→1	0.172	`Sqrt[2] + Sqrt[3]`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
L07	`Strad[Sqrt[1000003 + 2* Sqrt[999983]]]`	2→2	4.292	2→2	1.527	`Sqrt[1000003 + 2*Sqrt[ 999983]]`
L08	`Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]`	2→2	2.033	2→2	1.036	`Sqrt[2 + Sqrt[3] + Sqrt[ 5]]`
L09	`Strad[(5 + 2*Sqrt[6])^(1/ 6)]`	2→2	17.495	2→2	26.219	`(5 + 2*Sqrt[6])^(1/6)`
L10	`Strad[(11*Sqrt[2] + 9* Sqrt[3])^(1/3)]`	2→1	0.048	2→1	0.035	`Sqrt[2] + Sqrt[3]`
L11	`Strad[Expand[(2^(1/3) + 3^(1/3))^3]^(1/3)]`	2→1	0.094	2→1	0.057	`2^(1/3) + 3^(1/3)`
L12	`Strad[Sqrt[Expand[(1 + 2^ (1/3) + 3^(1/3))^2]]]`	2→1	0.070	2→1	0.055	`1 + 2^(1/3) + 3^(1/3)`
L13	`Strad[Expand[(1 + 2^(1/5) )^5]^(1/5)]`	2→1	0.081	2→1	0.047	`1 + 2^(1/5)`
L14	`Strad[Sqrt[5 + 2*Sqrt[6]] ^(1/5)]`	2→2	1.838	2→2	1.073	`(5 + 2*Sqrt[6])^(1/10)`
Q01	`Strad[Expand[(-1 - Sqrt[ 2])^3]^(1/3)]`	2→1	0.066	2→1	0.033	`(-1)^(1/3)*(1 + Sqrt[2])`
Q02	`Strad[Expand[(1 - Sqrt[2] )^3]^(1/3)]`	2→1	0.062	2→1	0.034	`(-1)^(1/3)*(-1 + Sqrt[2] )`
Q07	`Strad[Expand[(2 - 3*Sqrt[ 2])^3]^(1/3)]`	2→1	0.068	2→1	0.036	`(-1)^(1/3)*(-2 + 3*Sqrt[ 2])`
Q08	`Strad[(-7 - 5*Sqrt[2])^( 1/3)]`	2→1	0.061	2→1	0.030	`(-1)^(1/3)*(1 + Sqrt[2])`

6.3 The round-3 cases

Table 8 runs the inputs that the three reviews used to motivate their findings, plus controls, through both versions (`harness/cases_new.wl`; `exp_new2.wl` for the previous version). All 33 outputs of `StradFixed3.wl` are exactly equal to their inputs. The four three-surd coset identities N01–N04 and the rational-coefficient case N05 are found by the fast path in 0.1–0.3 s (N06 and N07 repeat two of them with `"MaxTrials" -> 0`: the previous version returns them unchanged, the new one denests them); the previous version needed the multiplier search, between 0.3 and 5.5 s, and N33, a five-surd square, took 31.8 s against 0.27 s. The odd-index rows N08–N14 and the Honsbeek rows N16–N17 are found by both versions; N15 asks for `"MaxOddIndex" -> 11`, which the previous version rejects as an unknown option (the **WRONG** mark of its column records that the returned `Failure` is not equal to the expected value), and which lets the new one denest an eleventh root without the search. The index reductions N18–N20 take 2–4 s now against 3–10 s. N26 shows the transcendental `Root` object left unchanged by both versions and N27 shows it left unchanged as a host while the radical next to it is denested; N28 is the malformed call, unevaluated before and a `Failure` now. The controls N29–N32 behave as before.

Table 8: Round-3 cases (N): the reviews' motivating inputs and the unified-C controls, `StradFixed2` against `StradFixed3`. A $\times$ after a depth marks an output not exactly equal to the expected value; the two occurrences are a rejected unknown option (N15, previous version) and the rejected malformed call N28.

ID	Input	StradFixed2		StradFixed3		Output of StradFixed3
		depth	time (s)	depth	time (s)	
N01	`Strad[Sqrt[118 + 2*Sqrt[210] + 14*Sqrt[55] + 2*Sqrt[462]]]`	2→1	5.528	2→1	0.259	`Sqrt[6] + Sqrt[35] + Sqrt[77]`
N02	`Strad[Sqrt[37 + 4*Sqrt[15] + 6*Sqrt[14] + 2*Sqrt[210]]]`	2→1	1.224	2→1	0.172	`Sqrt[6] + Sqrt[10] + Sqrt[21]`
N03	`Strad[Sqrt[60 + 10*Sqrt[6] + 10*Sqrt[14] + 10*Sqrt[21]]]`	2→1	0.848	2→1	0.115	`Sqrt[10] + Sqrt[15] + Sqrt[35]`
N04	`Strad[Sqrt[142 + 28*Sqrt[15] + 20*Sqrt[21] + 12*Sqrt[35]]]`	2→1	0.739	2→1	0.160	`Sqrt[30] + Sqrt[42] + Sqrt[70]`
N05	`Strad[Sqrt[(5 + Sqrt[6] + Sqrt[10] + Sqrt[15])/2]]`	2→1	0.307	2→1	0.082	`(Sqrt[2] + Sqrt[3] + Sqrt[5])/2`
N06	`Strad[Sqrt[118 + 2*Sqrt[210] + 14*Sqrt[55] + 2*Sqrt[462]], "MaxTrials" -> 0]`	2→2	0.988	2→1	0.228	`Sqrt[6] + Sqrt[35] + Sqrt[77]`
N07	`Strad[Sqrt[142 + 28*Sqrt[15] + 20*Sqrt[21] + 12*Sqrt[35]], "MaxTrials" -> 0]`	2→2	0.341	2→1	0.138	`Sqrt[30] + Sqrt[42] + Sqrt[70]`
N08	`Strad[(41 + 29*Sqrt[2])^(1/5)]`	2→1	0.060	2→1	0.013	`1 + Sqrt[2]`
N09	`Strad[(41 - 29*Sqrt[2])^(1/5)]`	2→1	0.292	2→1	0.078	`(-1)^(1/5)*(-1 + Sqrt[2])`
N10	`Strad[(239 + 169*Sqrt[2])^(1/7)]`	2→1	0.119	2→1	0.012	`1 + Sqrt[2]`
N11	`Strad[(-239 - 169*Sqrt[2])^(1/7)]`	2→1	0.491	2→1	0.115	`(-1)^(1/7)*(1 + Sqrt[2])`
N12	`Strad[(1393 - 985*Sqrt[2])^(1/9)]`	2→1	0.261	2→1	0.073	`(-1)^(1/9)*(-1 + Sqrt[2])`
N13	`Strad[Expand[(2 + Sqrt[3])^9]^(1/9)]`	2→1	0.065	2→1	0.014	`2 + Sqrt[3]`
N14	`Strad[Expand[(1 + Sqrt[5])^11]^(1/11)]`	2→1	0.091	2→1	0.063	`1 + Sqrt[5]`
N15	`Strad[Expand[(1 + Sqrt[5])^11]^(1/11), "MaxOddIndex" -> 11]`	2→0 $\times$	0.000	2→1	0.020	`1 + Sqrt[5]`
N16	`Strad[Sqrt[28^(1/3) - 3]]`	2→1	0.154	2→1	0.066	`(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3`
N17	`Strad[Sqrt[5^(1/3) - 4^(1/3)]]`	2→1	0.333	2→1	0.491	`(2^(1/3) + 2^(2/3)*5^(1/3) - 5^(2/3))/3`
N18	`Strad[(3 + 2*Sqrt[2])^(1/4)]`	2→2	7.235	2→2	2.229	`Sqrt[1 + Sqrt[2]]`

ID	Input	StradFixed2 depth	StradFixed2 time (s)	StradFixed3 depth	StradFixed3 time (s)	Output of StradFixed3
N19	`Strad[(3 + 2*Sqrt[2])^(1/6)]`	2→2	10.128	2→2	4.141	`(1 + Sqrt[2])^(1/3)`
N20	`Strad[(3 + 2*Sqrt[2])^(1/6), "MaxRecursion" -> 0]`	2→2	2.621	2→2	2.706	`(1 + Sqrt[2])^(1/3)`
N21	`Strad[(7*20^(1/3) - 19)^(1/6)]`	2→1	0.945	2→1	0.369	`-(2/3)^(1/3) + (5/3)^(1/3)`
N22	`Strad[(133 + 57*2^(2/3)*3^(1/3) + 48*2^(1/3)*3^(2/3))^(1/6)]`	2→1	0.197	2→1	0.065	`2^(1/3) + 3^(1/3)`
N23	`Strad[Sqrt[1 + Sqrt[3]] + Sqrt[3 + 3*Sqrt[3]] - Sqrt[10 + 6*Sqrt[3]]]`	2→0	3.721	2→0	1.767	`0`
N24	`Strad[1/(Sqrt[2] + Sqrt[3 + 2*Sqrt[2]])]`	2→1	0.057	2→1	0.030	`(1 + 2*Sqrt[2])^(-1)`
N25	`Strad[Sqrt[-(16 - 2*Sqrt[29] + 2*Sqrt[55 - 10*Sqrt[29]])]]`	3→2	3.362	3→2	2.460	`I*(Sqrt[5] + Sqrt[11 - 2*Sqrt[29]])`
N26	`Strad[Root[#1^5 + #1 - Pi & , 1]]`	0→0	0.005	0→0	0.003	`Root[-Pi + #1 + #1^5 & , 1]`
N27	`Strad[Root[#1^5 + #1 - 1 & , 1] + Sqrt[5 + 2*Sqrt[6]]]`	2→2	0.527	2→2	0.685	`Sqrt[2] + Sqrt[3] + (-1 + ((25 - 3*Sqrt[69])/2)^(1/3) + ((25 + 3*Sqrt[69])/2)^(1/3))/3`
N28	`Strad[Sqrt[2], 17]`	1→1	0.000	1→0	0.000	`Failure["InvalidOption", <|"MessageTemplate" -> "Options must be rules.", "Rules" -> {17}|>]`
N29	`Strad[Sqrt[1 + Sqrt[2]]]`	2→2	1.458	2→2	0.865	`Sqrt[1 + Sqrt[2]]`
N30	`Strad[Sqrt[2 + Sqrt[3] + Sqrt[5]]]`	2→2	6.937	2→2	2.445	`Sqrt[2 + Sqrt[3] + Sqrt[5]]`
N31	`Strad[Sqrt[2 + Sqrt[3]] + Sqrt[5 + 2*Sqrt[6]], True]`	2→1	0.440	2→1	0.112	`Sqrt[3/2] + 3/Sqrt[2] + Sqrt[3]`
N32	`Strad[Expand[(Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7])^2]^(1/2)]`	2→1	1.301	2→1	1.123	`Sqrt[2] + Sqrt[3] + Sqrt[5] + Sqrt[7]`
N33	`Strad[Expand[(Sqrt[6] + Sqrt[10] + Sqrt[15] + Sqrt[21] + Sqrt[35])^2]^(1/2)]`	2→1	31.802	2→1	0.272	`Sqrt[6] + Sqrt[10] + Sqrt[15] + Sqrt[21] + Sqrt[35]`

6.4 Randomized families

The ten structured families of unified-A, with the same seed `SeedRandom[20260905]` and the same 60-second limit per input, plus four new families for the round-3 methods, were run on the new version (Table 9). All 430 inputs were processed in 85 s; 0 results differ from the true value; no input reached its 60-second limit; every constructed identity (families F1, F2, F4–F14) is denested, and the generic family F3 yields the same six denestings and the same 34 unchanged or equal-rewritten controls as before. The three-surd squares of F8 and the products of F10 are four times faster than before because the coset search replaces the multiplier search; the new families F11–F14 exercise the coset search with mixed signs and composite radicands and the odd-index recipe for $q = 5, 7$ and

the even index 6. The first run of the new families exposed C22: with a negative coefficient in the radicand, six inputs of F8, F10 and F11 took 12–30 s each (`harness/repro_fuzz_slow.wl` replays the random inputs and times them one by one; after the fix no input of these families exceeds two seconds, log `repro_fuzz_slow.txt`).

		StradFixed2			StradFixed3			StradFixed3 time (s)	
Family	n	denested	unchanged	**wrong**	denested	unchanged	**wrong**	total	max
F1	40	40	0	0	40	0	0	0.683	0.041
F2	30	30	0	0	30	0	0	1.565	0.120
F3	40	6	34	0	6	34	0	50.572	3.347
F4	40	40	0	0	40	0	0	2.321	0.103
F5	40	40	0	0	40	0	0	3.452	0.173
F6	40	40	0	0	40	0	0	6.080	0.262
F7	25	25	0	0	25	0	0	1.406	0.148
F8	25	25	0	0	25	0	0	1.444	0.171
F9	30	30	0	0	30	0	0	0.730	0.037
F10	30	30	0	0	30	0	0	2.260	0.113
F11	30	–	–	–	30	0	0	3.941	0.303
F12	25	–	–	–	25	0	0	4.184	0.526
F13	20	–	–	–	20	0	0	4.971	0.519
F14	15	–	–	–	15	0	0	1.798	0.363
Total	430	306	34	0	396	34	0	85.408	3.347

Table 9: Randomized families: second corrected version (from unified-B) against `StradFixed3`. "unchanged" includes equal rewritten outputs. F1 $\sqrt{(p+q\sqrt{c})^2}$; F2 $((p+q\sqrt{c})^3)^{1/3}$; F3 $\sqrt{a+b\sqrt{c}}$ with random a; F4 $\sqrt{(p+qi\sqrt{c})^2}$; F5 $((p+qi\sqrt{c})^2)^{3/2}$; F6 $\sqrt{u^2}\sqrt{v^2}$ with complex u, v; F7 $((p+q\sqrt{c})^4)^{1/4}$; F8 $\sqrt{(p+q\sqrt{c}+r\sqrt{d})^2}$; F9 $((p+q\sqrt{c})^2)^{3/2}$, real; F10 $\sqrt{-u^2}\sqrt{-v^2}$, real; F11 $\sqrt{(\pm\sqrt{a}\pm\sqrt{b}\pm\sqrt{c})^2}$, three surds; F12 $((p+q\sqrt{c})^5)^{1/5}$; F13 $((p+q\sqrt{c})^7)^{1/7}$; F14 $((p+q\sqrt{c})^6)^{1/6}$.

6.5 The fifteen previously missed inputs

The four probe scripts of the probing session were rerun against `StradFixed3.wl` (the four scripts `harness/probe_gaps3_*.wl`, transcripts of the same names in `logs/`). All fifteen inputs of `KNOWN_GAPS.md` are denested to the expected depth, in between 0.06 s and 3.3 s (the slowest is Landau's cancellation sum Q21, which reduces to 0; the five-surd square root P13 takes 2.0 s). The controls and kernel-evaluated inputs of the probe scripts give the same outcomes as in unified-B (the `non-ok` lists of the four logs are identical to those of `unified-B/logs/`).

6.6 The regression suite

`tests/StradFixed3.wlt` contains 230 `VerificationTests`: the 144 tests of unified-B (the translated suites of review-4/5/6, the fifteen gaps, the unified-A contracts) rebound to the new context, and 86 new tests: the 50 of R7, the 52 of R8 and the 40 of R9 translated, merged where they coincide, onto the names and arities of `StradFixed3.wl`, plus seventeen controls of this document. The translation maps R8's `surdSystem`, `cosetBases`, `honsbeekSquareRoots` and `quadraticOddRoots` literally; replaces options that do not exist here (`"SquareClassSearch"`, `"ListProgress"`, `"ProcessExactIslands"`, `"MaxCharacterRank"`) by validation tests of the new options; replaces R9's `no-negative-cache` (two identical visits, two searches) by the budget-aware contract (a weak search followed by two full visits, two searches); rewrites two tests whose premise the kernel defeats (`Sqrt[-rho]` for a positive sum evaluates to `I Sqrt[rho]`, so the negative-radicand stage is exercised on $(-7-5\sqrt{2})^{1/3}$ instead; $(3+2\sqrt{2})^{1/4} \to \sqrt{1+\sqrt{2}}$ is an improvement in cost, not in depth); and drops R8's two opt-in policy tests. Two tests failed on the first run against `StradFixed3.wl` and led to fixes: R7's throwing solver (C20) and R8's grammar test with `Root[##^3 + Sqrt[2] ## + 1 &, 1]`, which exposed the triangular-system `Root` form

(C01). `tests/run_tests.wls` runs the suite in a fresh kernel: 230 of 230 tests succeed and 0 fail, in 28 s (log `regression_suite.txt`).

6.7 The reviews' own suites against their own proposals

None of the reviews could execute its own proposed implementation. The runner script `run_review_suite3.wl` in `harness/` loads each proposal in a fresh kernel and runs the review's own suite unchanged (logs `review*_suite.txt`). R7: 49 of 50 tests pass; the failure is `throwing-solver-quarantined`, whose own proposal has the `Catch[..., _]` defect of C20 (an untagged `Throw` from the solver escapes and the call returns `Hold[Throw[0]]`). R8: 52 of 52 tests pass. R9: 38 of 40 pass; its character-depth-reduction test fails because its character-reconstruction kernel returns the input $\sqrt{37+4\sqrt{15}+6\sqrt{14}+2\sqrt{210}}$ at depth 2 with `"MaxTrials" -> 0` (its character-missing-coset test passes only because `CertifiedEqualQ` of an unchanged result is `True`), and its partial-negative-improvement test fails because the kernel evaluates `Sqrt[-rho]` for the positive sum ρ to `I Sqrt[rho]`, so the helper under test never sees a negative radicand (the same premise had to be rewritten for the suite of Section 6.6). All three proposals load without messages in Wolfram 15.0.1, which the reviews could not establish. The suites are modest in coverage and none of them measures time; R8's coset search is off by default in its proposal and its suite tests the helpers directly.

6.8 Review-8's differential corpus

R8 supplied `tests/differential.wls`, an 18-input corpus to be run against the previous and the proposed version. `harness/differential3.wl` runs it against `StradFixed2.wl` and `StradFixed3.wl` in one kernel with a 60 s budget each (Table 10). All 18 outputs of both versions are exactly equal to their inputs and reach the same depth; the new version takes 6.8 s for the corpus against 12.0 s, the largest differences being the coset identity (0.24 s against 2.2 s) and $(3+2\sqrt{2})^{1/6}$ (3.4 s against 6.0 s).

Table 10: The differential corpus of review-8 (`tests/differential.wls`), run natively against `StradFixed2` and `StradFixed3` with a 60 s budget. A **NO** after a depth would mark an output not exactly equal to its input.

Input	StradFixed2: depth, time (s), output			StradFixed3: depth, time (s), output		
`Sqrt[3 + 2*Sqrt[2]]`	1	0.060	`1 + Sqrt[2]`	1	0.010	`1 + Sqrt[2]`
`Sqrt[5 + 2*Sqrt[6]]`	1	0.020	`Sqrt[2] + Sqrt[3]`	1	0.020	`Sqrt[2] + Sqrt[3]`
`Sqrt[4 + 3*Sqrt[2]]`	1	0.020	`2^(1/4) + 2^(3/4)`	1	0.010	`2^(1/4) + 2^(3/4)`
`Sqrt[3/2 + Sqrt[3]]`	1	0.020	`(3^(1/4) + 3^(3/4))/2`	1	0.030	`(3^(1/4) + 3^(3/4))/2`
`(7 + 5*Sqrt[2])^(1/3)`	1	0.010	`1 + Sqrt[2]`	1	0.010	`1 + Sqrt[2]`
`(41 - 29*Sqrt[2])^(1/5)`	1	0.150	`-(-1)^(1/5) + (-1)^(1/5)*Sqrt[2]`	1	0.080	`(-1)^(1/5)*(-1 + Sqrt[2])`
`(-99 - 70*Sqrt[2])^(1/6)`	1	0.170	`(-1)^(1/6) + (-1)^(1/6)*Sqrt[2]`	1	0.120	`(-1)^(1/6)*(1 + Sqrt[2])`
`(-239 - 169*Sqrt[2])^(1/7)`	1	0.240	`(-1)^(1/7) + (-1)^(1/7)*Sqrt[2]`	1	0.110	`(-1)^(1/7)*(1 + Sqrt[2])`
`(1393 - 985*Sqrt[2])^(1/9)`	1	0.150	`-(-1)^(1/9) + (-1)^(1/9)*Sqrt[2]`	1	0.070	`(-1)^(1/9)*(-1 + Sqrt[2])`
`(-19 + 7*2^(2/3)*5^(1/3))^(1/6)`	1	0.360	`-(2/3)^(1/3) + (5/3)^(1/3)`	1	0.410	`-(2/3)^(1/3) + (5/3)^(1/3)`

Input	StradFixed2: depth, time (s), output			StradFixed3: depth, time (s), output		
`(133 + 57*2^(2/3)*3^(1/3) + 48*2^(1/3)*3^(2/3))^(1/6)`	1	0.080	`2^(1/3) + 3^(1/3)`	1	0.070	`2^(1/3) + 3^(1/3)`
`Sqrt[-2^(2/3) + 5^(1/3)]`	1	0.190	`(2^(1/3) + 2^(2/3)*5^(1/3) - 5^(2/3))/3`	1	0.420	`(2^(1/3) + 2^(2/3)*5^(1/3) - 5^(2/3))/3`
`Sqrt[-3 + 2^(2/3)*7^(1/3)]`	1	0.100	`(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3`	1	0.070	`(-1 - 2^(2/3)*7^(1/3) + 2^(1/3)*7^(2/3))/3`
`Sqrt[118 + 14*Sqrt[55] + 2*Sqrt[210] + 2*Sqrt[462]]`	1	2.240	`Sqrt[6] + Sqrt[35] + Sqrt[77]`	1	0.240	`Sqrt[6] + Sqrt[35] + Sqrt[77]`
`Sqrt[1 + Sqrt[3]] + Sqrt[3 + 3*Sqrt[3]] - Sqrt[10 + 6*Sqrt[3]]`	0	1.890	`0`	0	1.480	`0`
`(3 + 2*Sqrt[2])^(1/6)`	2	5.950	`(1 + Sqrt[2])^(1/3)`	2	3.360	`(1 + Sqrt[2])^(1/3)`
`Sqrt[2 + Sqrt[2]]`	2	0.310	`Sqrt[2 + Sqrt[2]]`	2	0.320	`Sqrt[2 + Sqrt[2]]`
`(Sqrt[2] + Sqrt[3 + 2*Sqrt[2]])^(-1)`	1	0.020	`(1 + 2*Sqrt[2])^(-1)`	1	0.020	`(1 + 2*Sqrt[2])^(-1)`

6.9 Timing

The classical identities take between 0.01 and 0.1 seconds each (Table 3), a little less than before because the coset search and the odd-index recipe replace multiplier trials, and because the patience rule now counts from any improvement (C23). Non-denestable inputs still run the multiplier search to its trial limit and take 0.5–3 s; the composite-index controls that need sub-island searches take 2–4 s, and the one non-denestable composite-index row of the battery, L09, takes 26 s, more than before, for the reasons given in Section 6.2. Every run stayed far below its `"TimeBudget"` of 120 seconds. The budget-aware memo was necessary: the positive-only memo the reviews asked for made $(3 + 2\sqrt{2})^{1/4}$ take 64 s and, with one recursion level, exhaust the budget.

7 Compatibility changes and remaining limitations

- **Grammar.** A `Root` object whose defining polynomial has coefficients outside the radical grammar, or an `AlgebraicNumber` with non-Gaussian coefficients, is no longer an exact algebraic island: it is an opaque host node, left unchanged, and `EqualityStatus` of it is `"Unknown"`. Genuinely algebraic numbers in such presentations are therefore conservatively rejected, as all three reviews recommended.
- **Equality.** `EqualityStatus` is exact in every branch. `"NumericPrefilter"` defaults to `False`; when `True` it prunes search candidates and can lose one, never accept one, and never changes a status. `CertificatesDifferent` counts exact rejections only.
- **Memo.** Negative results are reused only by searches of no larger budget; a top-level island is never answered by a recursive failure. Equal-budget reuse is kept for the reasons of Section 2.
- **Cost.** Gaussian components are priced and opaque counting is structural, so tie-breaks between equal outputs can change relative to `StradFixed2`.
- **Reports.** A disabled or interrupted report contains `Missing["NotComputed"]` costs; `ResultChanged` is separate from `Status`; the certificate list is labelled.

- **Dispatch.** Malformed calls return a `Failure` instead of staying unevaluated.
- **Policies kept.** Inexact hosts are left untouched; list progress is judged by the cost of the whole list; `"MaxTrials"` is per island; multiplier keys are syntactic.
- **Completeness.** There is none. The coset search is complete for rational combinations of square roots of positive integers only up to `"MaxCosets"` and the working precision of the relation stage (with the rational systems as a bounded fallback); the odd-index recipe covers real quadratic radicands up to `"MaxOddIndex"`; the Honsbeek recognizer needs exactly two terms; the multiplier family is heuristic; sums are still treated component by component except for the whole-island proposals.
- **Resources.** The kernel limits are cooperative. Argument evaluation before the call is outside the region. A pass interrupted by the deadline is lost; the committed result is the last complete pass.

8 Future work

1. **Commit partial work on interruption.** Publish the best certified island rewrites of an interrupted pass instead of the input; the gate already certifies each island, so the pass structure is a checkpoint, not a soundness requirement.
2. **Field-theoretic multiplier generation.** Replace the discriminant heuristic by the classes of $K^\times/(K^\times)^q$ for the extension at hand, as R7 and unified-B recommend.
3. **Integer relations beyond square roots.** The relation stage of the coset search generalizes to q-th roots over a Kummer basis; it would give a cheap proposal stage for sums of cube roots.
4. **Independent certificates.** Record defining polynomials, field maps and isolating intervals so that an accepted identity can be rechecked without `RootReduce`.
5. **A Pareto frontier** of certified alternatives per island, so that a shallower but longer form and a deeper but shorter form both survive until the containing expression decides.
6. **Cross-session caches** of minimal polynomials and factorizations keyed by canonical values, with the budget-aware status of the memo.

A Source of `StradFixed3.wl`

```
(* ::Package:: *)

(* StradFixed3.wl -- third corrected version of src/original/Strad.wl.

   Successor of StradFixed2.wl (RadicalDenest3'). It answers the three reviews
   of the second corrected version (src/code-review/review-7, review-8,
   review-9). The report in src/code-review/unified-C explains every change.

   Contract (for an exact algebraic input E and every exact algebraic island of
   a symbolic host):
     * every replaced island is certified equal to the island it replaces by
       exact algebra (RootReduce, then PossibleZeroQ with
       Method -> "ExactAlgebraics"), whatever produced the candidate; the
       optional numeric prefilter only prunes candidates inside the search and
       never decides an equality status;
     * an accepted replacement is strictly cheaper under RadicalCost, or the
       island is returned unchanged; every search stage returns its incumbent
       or a certified cheaper candidate;
     * the exact input grammar admits rationals, Gaussian rationals, Plus,
       Times and rational Power combinations of them, Root objects whose
       defining polynomial has coefficients in that grammar and a valid root
       index, and AlgebraicNumber objects with an admitted generator and
```

```
23          Gaussian-rational coefficients; other exact objects are unsupported;
24        * the whole call, including the classification of the input and the cost
25          computations of the report, runs inside one TimeConstrained and
26          MemoryConstrained region ("TimeBudget", "MemoryBudget"); every expensive
27          kernel operation runs inside its own bounded region; "MaxTrials" bounds
28          the multiplier trials spent on any one island;
29        * symbolic hosts are never simplified; ambient $Assumptions are ignored;
30        * malformed, unknown or invalid options produce a Failure.
31      There is no completeness or minimum-depth guarantee: an unchanged result
32      means that the enabled bounded methods found nothing cheaper. The kernel
33      limits are cooperative and are not an operating-system sandbox.
34
35      Context: RadicalDenest3`. The package can be loaded next to RadicalDenest3`
36      and RadicalDenest` for differential testing. Author: analysis session of
37      5 September 2026. *)
38
39  BeginPackage["RadicalDenest3`"];
40
41  Strad::usage = "Strad[expr, opts] denests the exact algebraic radicals occurring in expr and
        returns an expression certified equal to expr. Strad[expr, True, opts] also processes inner
        nesting levels and repeats passes while they improve the result.";
42  DenestRadicals::usage = "DenestRadicals[expr, opts] is the option-driven form of Strad with its
        own option defaults.";
43  DenestCore::usage = "DenestCore[problem, opts] denests one exact algebraic number without
        traversing a symbolic host.";
44  DenestReport::usage = "DenestReport[expr, opts] returns an Association with the result (\"
        Result\"), \"Status\", \"Limits\", \"Statistics\", \"Certificates\", \"ElapsedSeconds\", \"
        Options\" and an optional bounded \"Trace\".";
45  EqualityStatus::usage = "EqualityStatus[a, b] is \"Equal\", \"Different\" or \"Unknown\" for
        exact algebraic numbers a and b, decided by exact algebra only (RootReduce, then
        PossibleZeroQ with Method -> \"ExactAlgebraics\") within a time limit; inputs outside the
        supported grammar give \"Unknown\".";
46  CertifiedEqualQ::usage = "CertifiedEqualQ[a, b] is True only when EqualityStatus[a, b] is \"
        Equal\".";
47  ExactAlgebraicQ::usage = "ExactAlgebraicQ[e] is True when e is in the supported exact algebraic
        grammar: rationals, Gaussian rationals, Plus, Times and rational Power combinations of
        them, Root objects of a polynomial with such coefficients and a valid root index, and
        AlgebraicNumber objects with an admitted generator and Gaussian-rational coefficients.
        False also covers unsupported exact representations.";
48  RadicalExpressionQ::usage = "RadicalExpressionQ[e] is True when e is an explicit radical
        expression: rationals, Gaussian rationals and Plus, Times and rational Power combinations
        of them. Root and AlgebraicNumber objects are not radical expressions.";
49  RadicalDepth::usage = "RadicalDepth[e] is the maximal number of nested rational-power nodes on
        a path of e. Root and AlgebraicNumber objects are opaque and count as depth 0.";
50  RadicalCost::usage = "RadicalCost[e] is the lexicographic cost {#Root and AlgebraicNumber
        objects, radical depth, #rational-power nodes outside opaque objects, LeafCount, total bit
        size of integer, rational and Gaussian-rational atoms} used to decide whether a rewrite is
        an improvement.";
51  RationalizeDenominator::usage = "RationalizeDenominator[expr] rewrites expr with a rational
        denominator using a certified polynomial inverse of the exact algebraic denominator. It
        need not reduce RadicalCost.";
52  Factorc::usage = "Factorc[expr] offers Factor of an exact algebraic expr as a certified
        proposal and returns it only when it is certified equal and cheaper. Symbolic input is
        returned unchanged.";
53
54  Options[Strad] = {
55      "AllLevels" -> False, "Verbose" -> False, "Trace" -> False,
56      "Multipliers" -> Automatic, "Solver" -> Automatic, "Factor" -> True,
57      "MaxTrials" -> 120, "TimeBudget" -> 120, "OperationTime" -> 30,
58      "CertifyTime" -> 20, "MemoryBudget" -> 1073741824,
59      "MultiplierCap" -> 1000, "MaxRootIndex" -> 32, "MaxDegree" -> 64,
60      "MaxSolveDegree" -> 4, "MaxLeafCount" -> 20000, "MaxPasses" -> 4,
61      "MaxRecursion" -> 3, "Patience" -> 25, "NumericPrefilter" -> False, "MaxTraceEntries" -> 200,
62      "MaxOddIndex" -> 9, "DiscriminantBatchCap" -> 24, "MaxCosets" -> 16};
63  Options[DenestRadicals] = Options[Strad];
64  Options[DenestCore] = Options[Strad];
65  Options[DenestReport] = Options[Strad];
66
67  Begin["`Private`"];
68
69  (* ------------------------------------------------------------------ *)
```

```
(* session state (dynamically scoped by run[]), statistics, tracing   *)
(* ------------------------------------------------------------------ *)

$active = False;
$cfg = Association[Options[Strad]];
$deadline = Infinity;
$stats = <||>; $limits = <||>; $trace = {}; $records = {}; $memo = <||>; $inProgress = <||>;
$recursion = 0; $lastCertificateMethod = "None";
$x = Unique["RadicalDenest3`Private`x"];

newStats[] := <|"Islands" -> 0, "Trials" -> 0, "Operations" -> 0,
   "OperationTimeouts" -> 0, "OperationFailures" -> 0,
   "Certificates" -> 0, "CertificatesEqual" -> 0, "CertificatesDifferent" -> 0,
   "CertificatesUnknown" -> 0, "NumericRejections" -> 0, "CosetSystems" -> 0,
   "CandidatesAccepted" -> 0, "MultipliersProposed" -> 0,
   "MultipliersAdmitted" -> 0, "DuplicateMultipliers" -> 0,
   "FastPathAccepted" -> 0, "PassesCompleted" -> 0|>;

bump[key_String] := AssociateTo[$stats, key -> (Lookup[$stats, key, 0] + 1)];
limitHit[key_String] := AssociateTo[$limits, key -> True];
remaining[] := $deadline - AbsoluteTime[];
expiredQ[] := If[remaining[] <= 0, limitHit["TimeBudget"]; True, False];

log[args___] := If[TrueQ[$cfg["Verbose"]], Print["[RadicalDenest3] ", args]];
trace[tag_String, data_: <||>] := If[TrueQ[$cfg["Trace"]] &&
    Length[$trace] < $cfg["MaxTraceEntries"],
   AppendTo[$trace, <|"Event" -> tag, "Data" -> data|>]];

(* Every expensive kernel operation runs inside bounded[...]: its own time
   limit, clipped to the remaining session time, with messages silenced and
   failures turned into $Failed. *)
SetAttributes[bounded, HoldAll];
bounded[body_, kind_: "Operation"] := Module[{seconds, value},
   seconds = Min[remaining[], If[kind === "Certificate", $cfg["CertifyTime"], $cfg["
     OperationTime"]]];
   If[seconds <= 0,
    limitHit[If[remaining[] <= 0, "TimeBudget", kind <> "Time"]]; Return[$Failed]];
   bump["Operations"];
   value = Quiet[Check[TimeConstrained[body, seconds, operationTimedOut], operationFailed]];
   Which[
    value === operationTimedOut,
     bump["OperationTimeouts"]; trace["OperationTimeout"];
     limitHit[If[kind === "Certificate", "CertifyTime", "OperationTime"]];
     If[remaining[] <= 0, limitHit["TimeBudget"]]; $Failed,
    value === operationFailed || value === $Failed || value === $Aborted,
     bump["OperationFailures"]; $Failed,
    True, value]];

(* ------------------------------------------------------------------ *)
(* option resolution and validation                                   *)
(* ------------------------------------------------------------------ *)

finiteNonnegativeQ[v_] := MatchQ[v, _Integer | _Rational | _Real] && TrueQ[0 <= v < Infinity];
nonnegativeIntegerQ[v_] := IntegerQ[v] && v >= 0;
positiveIntegerQ[v_] := IntegerQ[v] && v >= 1;
booleanQ[v_] := v === True || v === False;

flattenRules[items_List] := Flatten[Replace[items, l_List :> flattenRules[l], {1}], 1];

resolveOptions[head_Symbol, raw_List] := Module[{rules, names, unknown, cfg, bad},
   rules = flattenRules[raw];
   If[! AllTrue[rules, MatchQ[#, _Rule | _RuleDelayed] &],
    Return[Failure["InvalidOption", <|"MessageTemplate" -> "Options must be rules.", "Rules" ->
      rules|>]]];
   names = First /@ Options[head];
   unknown = Complement[First /@ rules, names];
   If[unknown =!= {},
    Return[Failure["UnknownOption", <|"MessageTemplate" -> "Unknown option(s): `Keys`.", "Keys"
      -> unknown|>]]];
   (* effective defaults of the head actually called, first explicit rule wins *)
   cfg = Association[Table[With[{name = key}, name -> OptionValue[head, rules, name]], {key,
     names}]];
```

```
    bad = Join[
      Select[{"AllLevels", "Verbose", "Trace", "Factor", "NumericPrefilter"}, ! booleanQ[cfg[#]] &],
      Select[{"MaxTrials", "MultiplierCap", "MaxTraceEntries", "MaxRecursion", "Patience", "DiscriminantBatchCap", "MaxCosets"}, ! nonnegativeIntegerQ[cfg[#]] &],
      Select[{"MemoryBudget", "MaxRootIndex", "MaxDegree", "MaxSolveDegree", "MaxLeafCount", "MaxPasses", "MaxOddIndex"}, ! positiveIntegerQ[cfg[#]] &],
      Select[{"TimeBudget", "OperationTime", "CertifyTime"}, ! finiteNonnegativeQ[cfg[#]] &]];
    If[! (cfg["Multipliers"] === Automatic || ListQ[cfg["Multipliers"]]), AppendTo[bad, "Multipliers"]];
    If[! (cfg["Solver"] === Automatic || MatchQ[cfg["Solver"], _Function | _Symbol]), AppendTo[bad, "Solver"]];
    If[cfg["MaxSolveDegree"] > 4, AppendTo[bad, "MaxSolveDegree"]];
    If[bad =!= {},
     Return[Failure["InvalidOption", <|"MessageTemplate" -> "Invalid value(s) for `Keys`; limits must be finite and of the documented type.", "Keys" -> bad|>]]];
    cfg];

(* ------------------------------------------------------------------ *)
(* grammar, depth, cost                                               *)
(* ------------------------------------------------------------------ *)

rationalQ[e_] := MatchQ[e, _Integer | _Rational];
gaussianQ[e_] := rationalQ[e] || (Head[e] === Complex && rationalQ[Re[e]] && rationalQ[Im[e]]);
opaqueQ[e_] := MatchQ[e, _Root | _AlgebraicNumber];

RadicalExpressionQ[e_] := Which[
   gaussianQ[e], True,
   AtomQ[e], False,
   MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, RadicalExpressionQ],
   Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && RadicalExpressionQ[e[[1]]],
   True, False];

(* A Root object is admitted only when its defining function gives a nonconstant
   univariate polynomial whose coefficients are explicit algebraic numbers of the
   radical grammar and its index is a valid root selector; an AlgebraicNumber
   needs an admitted generator and Gaussian-rational coefficients. Exactness of
   the representation alone (Root[#^5 + # - Pi &, 1]) does not make a number
   algebraic. *)
rootPolynomial[e_] := Module[{args, fs, ks, p, degree, vars, ps},
   If[Head[e] =!= Root, Return[$Failed]];
   args = List @@ e;
   If[! MemberQ[{2, 3}, Length[args]], Return[$Failed]];
   If[Length[args] === 3 && ! MemberQ[{0, 1}, args[[3]]], Return[$Failed]];
   fs = args[[1]]; ks = args[[2]];
   Which[
    Head[fs] === Function && positiveIntegerQ[ks],
     p = Quiet[Check[Expand[fs[$x]], $Failed]];
     If[p === $Failed || ! TrueQ[PolynomialQ[p, $x]], Return[$Failed]];
     degree = Exponent[p, $x];
     If[! positiveIntegerQ[degree] || ks > degree, Return[$Failed]];
     If[! AllTrue[CoefficientList[p, $x], RadicalExpressionQ], Return[$Failed]];
     p,
    (* the kernel writes a root of a polynomial with algebraic coefficients as a
       triangular system Root[{f1, f2, ...}, {k1, k2, ...}]; every polynomial of
       the system must have coefficients in the radical grammar *)
    ListQ[fs] && ListQ[ks] && Length[fs] === Length[ks] && fs =!= {} && AllTrue[fs, Head[#] === Function &] && AllTrue[ks, positiveIntegerQ],
     vars = Table[Unique["RadicalDenest3`Private`rv"], {Length[fs]}];
     ps = Quiet[Check[Expand[#[Sequence @@ vars]] & /@ fs, $Failed]];
     If[ps === $Failed || ! AllTrue[ps, TrueQ[PolynomialQ[#, vars]] && ! FreeQ[#, Alternatives @@ vars] &], Return[$Failed]];
     If[! AllTrue[ps, AllTrue[Values[CoefficientRules[#, vars]], RadicalExpressionQ] &], Return[$Failed]];
     Last[ps],
    True, $Failed]];

algebraicFormQ[e_] := Which[
   gaussianQ[e], True,
   Head[e] === Root, rootPolynomial[e] =!= $Failed,
   Head[e] === AlgebraicNumber, Length[e] === 2 && ListQ[e[[2]]] && AllTrue[e[[2]], gaussianQ] && algebraicFormQ[e[[1]]],
```

```
    AtomQ[e], False,
    MemberQ[{Plus, Times}, Head[e]], AllTrue[List @@ e, algebraicFormQ],
    Head[e] === Power && Length[e] === 2, rationalQ[e[[2]]] && algebraicFormQ[e[[1]]],
    True, False];

(* exactness is decided by the grammar (algebraic by construction) *)
ExactAlgebraicQ[e_] := algebraicFormQ[e] && FreeQ[e, Indeterminate | ComplexInfinity |
     _DirectedInfinity];
exactQ[e_] := ExactAlgebraicQ[e];

RadicalDepth[e_] := Which[
    AtomQ[e] || opaqueQ[e], 0,
    MatchQ[e, Power[_, _Rational]], 1 + RadicalDepth[First[e]],
    True, Max[Prepend[RadicalDepth /@ (List @@ e), 0]]];

(* recursive traversals that stop at opaque objects; Gaussian atoms are priced
    through their components *)
radicalNodes[e_] := Which[opaqueQ[e] || AtomQ[e], 0,
    True, Boole[MatchQ[e, Power[_, _Rational]]] + Total[radicalNodes /@ (List @@ e)]];
opaqueCount[e_] := Which[opaqueQ[e], 1, AtomQ[e], 0, True, Total[opaqueCount /@ (List @@ e)]];
bitSize[e_] := Which[
    IntegerQ[e], 1 + IntegerLength[Abs[e], 2],
    Head[e] === Rational, 2 + IntegerLength[Abs[Numerator[e]], 2] + IntegerLength[Denominator[e],
       2],
    Head[e] === Complex, bitSize[Re[e]] + bitSize[Im[e]],
    AtomQ[e], 0,
    True, Total[bitSize /@ (List @@ e)]];

RadicalCost[e_] := {opaqueCount[e], RadicalDepth[e], radicalNodes[e], LeafCount[e], bitSize[e]};

cheaperQ[new_, old_] := Order[RadicalCost[new], RadicalCost[old]] === 1;
pickCheapest[list_List] := First[SortBy[list, RadicalCost]];
smallQ[e_] := If[LeafCount[e] > $cfg["MaxLeafCount"], limitHit["MaxLeafCount"]; False, True];

(* ---------------------------------------------------------------- *)
(* exact certification                                              *)
(* ---------------------------------------------------------------- *)

(* RootReduce is a canonicalizer: a reduced difference that is a nonzero exact
    algebraic number (an integer, rational, Gaussian rational, Root object or
    explicit radical form such as 2 Sqrt[2]) proves inequality *)
canonicalNonzeroQ[e_] := e =!= 0 && exactQ[e] && FreeQ[e, RootReduce];

(* heuristic pruning by significance arithmetic, used only by the search gate;
    it can lose a candidate, never accept one, and never decides EqualityStatus *)
numericallyDifferentQ[d_] := Module[{num},
    If[! TrueQ[$cfg["NumericPrefilter"]], Return[False]];
    num = bounded[N[d, 40]];
    TrueQ[NumberQ[num] && Accuracy[num] >= 25 && Abs[num] > 10^-20]];

certify[a_, b_] := Module[{d, r},
    bump["Certificates"]; $lastCertificateMethod = "None";
    If[! exactQ[a] || ! exactQ[b], bump["CertificatesUnknown"]; Return["Unknown"]];
    If[a === b, $lastCertificateMethod = "SameQ"; bump["CertificatesEqual"]; Return["Equal"]];
    d = a - b;
    r = bounded[RootReduce[d], "Certificate"];
    Which[
     r === 0, $lastCertificateMethod = "RootReduce"; bump["CertificatesEqual"]; Return["Equal"],
     canonicalNonzeroQ[r], $lastCertificateMethod = "RootReduce"; bump["CertificatesDifferent"];
       Return["Different"]];
    r = bounded[PossibleZeroQ[d, Method -> "ExactAlgebraics"], "Certificate"];
    Which[
     r === True, $lastCertificateMethod = "PossibleZeroQ/ExactAlgebraics"; bump["
      CertificatesEqual"]; "Equal",
     r === False, $lastCertificateMethod = "PossibleZeroQ/ExactAlgebraics"; bump["
      CertificatesDifferent"]; "Different",
     True, bump["CertificatesUnknown"]; "Unknown"]];

SetAttributes[standalone, HoldAll];
standalone[body_, failure_] := If[TrueQ[$active], body,
    Block[{$active = True, $cfg = Association[Options[Strad]], $deadline = AbsoluteTime[] + 20,
      $stats = newStats[], $limits = <||>, $trace = {}, $records = {}, $memo = <||>, $inProgress
```

```
      = <||>,
266       $recursion = 0, $lastCertificateMethod = "None", $Assumptions = True},
267      Quiet[Check[TimeConstrained[MemoryConstrained[body, 1073741824, failure], 20, failure],
          failure]]]];
268
269  EqualityStatus[a_, b_] := standalone[certify[a, b], "Unknown"];
270  CertifiedEqualQ[a_, b_] := EqualityStatus[a, b] === "Equal";
271
272  (* The single acceptance gate. A candidate replaces the incumbent only if it is
273     an explicit radical expression, small, strictly cheaper than the incumbent,
274     and certified equal to the ORIGINAL target of this island. *)
275  accept[candidate_, target_, incumbent_, method_String] := Module[{},
276     If[candidate === $Failed || candidate === target || ! RadicalExpressionQ[candidate] ||
277       ! smallQ[candidate] || ! cheaperQ[candidate, incumbent], Return[incumbent]];
278     (* optional heuristic pruning: skips the exact certificate of a candidate
279        whose difference from the target is numerically far from zero *)
280     If[numericallyDifferentQ[candidate - target], bump["NumericRejections"]; Return[incumbent]];
281     If[certify[candidate, target] === "Equal",
282      bump["CandidatesAccepted"];
283      If[Length[$records] < $cfg["MaxTraceEntries"],
284       AppendTo[$records, <|"Before" -> target, "After" -> candidate, "Method" -> method,
285         "EqualityMethod" -> $lastCertificateMethod, "Scope" -> "AcceptedProposal"|>]];
286      log[method, ": ", target, " -> ", candidate];
287      trace["Accepted", <|"Method" -> method, "Cost" -> RadicalCost[candidate]|>];
288      candidate,
289      incumbent]];
290
291  (* variants of a candidate that may print more cheaply; each is re-gated *)
292  acceptWithPolish[candidate_, target_, incumbent_, method_String] := Module[{best = incumbent, v
         },
293     If[expiredQ[] || candidate === $Failed || ! smallQ[candidate] || ! exactQ[candidate], Return
          [best]];
294     best = accept[candidate, target, best, method];
295     v = bounded[Simplify[candidate, Assumptions -> True, TimeConstraint -> 5]];
296     best = accept[v, target, best, method <> "/Simplify"];
297     v = bounded[Expand[candidate]];
298     best = accept[v, target, best, method <> "/Expand"];
299     v = bounded[Together[candidate]];
300     best = accept[v, target, best, method <> "/Together"];
301     v = rationalizeRaw[candidate];
302     best = accept[v, target, best, method <> "/Rationalize"];
303     best];
304
305  (* ------------------------------------------------------------------ *)
306  (* certified denominator inverse                                      *)
307  (* ------------------------------------------------------------------ *)
308
309  validPolynomialQ[p_, x_, minimum_Integer: 1] := Module[{degree},
310     If[! FreeQ[p, $Failed | $Aborted | operationFailed | operationTimedOut] || ! TrueQ[
          PolynomialQ[p, x]], Return[False]];
311     degree = Exponent[p, x];
312     IntegerQ[degree] && degree >= minimum && degree <= $cfg["MaxDegree"]];
313
314  rationalizeRaw[e_] := Module[{t, num, den, p, c, inv, result},
315     t = bounded[Together[e]];
316     If[t === $Failed, Return[$Failed]];
317     num = Numerator[t]; den = Denominator[t];
318     If[rationalQ[den], Return[t]];
319     If[! exactQ[den] || ! smallQ[den], Return[$Failed]];
320     p = bounded[MinimalPolynomial[den, $x]];
321     If[! validPolynomialQ[p, $x], Return[$Failed]];
322     c = CoefficientList[p, $x];
323     If[! AllTrue[c, rationalQ] || First[c] === 0, Return[$Failed]];
324     (* p(d) = 0 gives 1/d = -(a1 + a2 d + ... + an d^(n-1))/a0 (Horner form) *)
325     inv = bounded[-Fold[#1 den + #2 &, Last[c], Reverse[Rest[Most[c]]]]/First[c]];
326     If[inv === $Failed || certify[den inv, 1] =!= "Equal", Return[$Failed]];
327     result = bounded[Expand[num inv]];
328     If[result === $Failed || ! smallQ[result], $Failed, result]];
329
330  RationalizeDenominator[e_] := standalone[Replace[rationalizeRaw[e], $Failed -> e], e];
331
332  Factorc[e_] := standalone[If[exactQ[e], accept[bounded[Factor[e]], e, e, "Factor"], e], e];
```

```
(* ------------------------------------------------------------------ *)
(* exact low-degree fast paths                                        *)
(* ------------------------------------------------------------------ *)

(* a + b Sqrt[c] with rational a, b, c > 0 nonsquare, after expansion *)
quadraticParts[rho_] := Module[{e, terms, rat, irr, a, t, c, b},
   e = bounded[Expand[rho]];
   If[e === $Failed, Return[$Failed]];
   terms = If[Head[e] === Plus, List @@ e, {e}];
   rat = Select[terms, rationalQ]; irr = Select[terms, ! rationalQ[#] &];
   If[Length[irr] =!= 1, Return[$Failed]];
   a = Total[rat]; t = First[irr];
   c = Replace[t, {Sqrt[cc_?rationalQ] :> cc, Times[k_?rationalQ, Sqrt[cc_?rationalQ]] :> k^2
     cc, _ -> $Failed}];
   If[c === $Failed || ! TrueQ[c > 0] || rationalQ[Sqrt[c]], Return[$Failed]];
   b = Replace[t, {Sqrt[_] :> 1, Times[k_?rationalQ, Sqrt[_]] :> Sign[k], _ -> 1}];
   (* represent as a + b Sqrt[c] with b = +-1 and c absorbing the coefficient *)
   {a, b, c}];

(* direct and indirect denesting of Sqrt[a + b Sqrt[c]] > 0 *)
quadraticSquareRoots[{a_, b_, c_}] := Module[{d, s, u, v, e, out = {}},
   d = a^2 - b^2 c;
   If[d >= 0,
    s = Sqrt[d];
    If[rationalQ[s] && a > 0,
     u = (a + s)/2; v = (a - s)/2;
     If[u >= 0 && v >= 0, AppendTo[out, Sqrt[u] + Sign[b] Sqrt[v]]]]];
   (* indirect criterion: -c d must be a rational square *)
   If[d < 0 && b > 0,
    e = Sqrt[b^2 - a^2/c];
    If[rationalQ[e] && 0 <= e <= b,
     AppendTo[out, c^(1/4) (Sqrt[(b + e)/2] + Sign[a] Sqrt[(b - e)/2])]]];
   out];

(* real odd roots inside Q(Sqrt[c]): trace-norm criterion with the Dickson
   polynomials D_q(T, n) = u^q + v^q and S_q(T, n) = (u^q - v^q)/(u - v) for
   u + v = T, u v = n.  beta = A + B Sqrt[c] with beta^q = a + b Sqrt[c] exists
   in Q(Sqrt[c]) iff n^q = a^2 - b^2 c and D_q(T, n) = 2 a have rational
   solutions; then S_q(T, n) != 0 and beta = T/2 + b Sqrt[c]/S_q(T, n). *)
quadraticOddRoots[{a_, b_, c_}, q_Integer] := Module[{norm, n, d0 = 2, d1 = $x, s0 = 0, s1 = 1,
     dn, sn, roots, den, beta, out = {}},
   If[! OddQ[q] || q < 3 || q > $cfg["MaxOddIndex"], Return[{}]];
   norm = a^2 - b^2 c;
   n = Sign[norm] Abs[norm]^(1/q);
   If[! rationalQ[n], Return[{}]];
   Do[dn = Expand[$x d1 - n d0]; sn = Expand[$x s1 - n s0];
    {d0, d1} = {d1, dn}; {s0, s1} = {s1, sn}, {q - 1}];
   roots = rationalRoots[d1 - 2 a];
   Do[den = s1 /. $x -> t;
    If[den =!= 0, beta = t/2 + b Sqrt[c]/den;
     If[TrueQ[bounded[RootReduce[beta^q - (a + b Sqrt[c])] === 0, "Certificate"]], AppendTo[out,
      beta]]],
    {t, roots}];
   out];
cubicInQuadratic[parts_List] := quadraticOddRoots[parts, 3];

rationalRoots[p_] := Module[{fl},
   fl = bounded[FactorList[p]];
   If[fl === $Failed || ! ListQ[fl], Return[{}]];
   DeleteDuplicates[Select[Cases[fl, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x]
     === 1 :>
       -Coefficient[f, $x, 0]/Coefficient[f, $x, 1]], rationalQ]]];

(* Honsbeek: Sqrt[A + B] with A^3, B^3 nonzero rationals, A + B > 0 real *)
(* Honsbeek: Sqrt[A + B] where A^3 and B^3 are nonzero rationals (a rational
   summand counts as its own cube root); both signs are offered and a negative
   denominator gives complex candidates, the gate selects the principal one *)
honsbeekSquareRoots[rho_] := Module[{terms, a, b, ratio, roots, den, num, out = {}},
   If[Head[rho] =!= Plus || Length[rho] =!= 2, Return[{}]];
   terms = List @@ rho;
   (* each summand must be a rational or a real cube-root-like term such as
```

```
401       28^(1/3) = 2^(2/3) 7^(1/3): depth one, radical indices dividing 3, cube rational *)
402     If[! AllTrue[terms, RadicalDepth[#] <= 1 && FreeQ[#, Complex] && FreeQ[#, Power[_,
          r_Rational /; ! IntegerQ[3 r]]] &], Return[{}]];
403     If[AllTrue[terms, rationalQ], Return[{}]];
404     {a, b} = Replace[bounded[RootReduce[#^3], "Certificate"], $Failed -> Null] & /@ terms;
405     If[! AllTrue[{a, b}, rationalQ] || a === 0 || b === 0, Return[{}]];
406     ratio = b/a;
407     roots = rationalRoots[$x^4 + 4 $x^3 + 8 ratio $x - 4 ratio];
408     Do[den = b - s^3 a;
409      If[den =!= 0,
410       num = -s^2 terms[[1]]^2/2 + s terms[[1]] terms[[2]] + terms[[2]]^2;
411       out = Join[out, {num/Sqrt[den], -num/Sqrt[den]}]],
412      {s, roots}];
413     out];
414
415   (* Sqrt of a rational combination of several square roots of integers.
416      The candidate Sum[x_u Sqrt[u], u in U] is solved for rational x_u; the
417      unknown sets U are (i) 1 and the primes of the radicands, (ii) also the
418      radicands, and (iii) the cosets d H of the square-class group H generated by
419      the radicands, d ranging over the classes generated by the primes of the
420      radicands and of the coefficients (at most "MaxCosets" cosets).  By the
421      single-coset theorem, a square root of rho that is a rational combination of
422      square roots of integers has its support in one such coset. *)
423   squarefreeClass[n_Integer] := Module[{fl},
424     fl = bounded[FactorInteger[Abs[n]]];
425     If[fl === $Failed || ! ListQ[fl], Return[$Failed]];
426     Times @@ (#[[1]]^Mod[#[[2]], 2] & /@ fl)];
427
428   (* the primes dividing any of the integers (signs and units ignored) *)
429   classPrimes[classes_List] := Module[{fl},
430     fl = bounded[Union @@ (Select[First /@ FactorInteger[Abs[#]], PrimeQ] & /@ DeleteCases[
          classes, 0 | 1 | -1])];
431     If[fl === $Failed || ! ListQ[fl], {}, fl]];
432
433   (* the coset unknown sets: each is a sorted list of squarefree integers *)
434   cosetBases[radicands_List, primes_List] := Module[{vec, classOf, hVectors, h, all, cosets = {},
          seen = <||>, rep},
435     If[primes === {} || Length[primes] > 10, Return[{}]];
436     vec[n_] := Boole[Divisible[n, #]] & /@ primes;
437     classOf[v_] := Times @@ MapThread[Power, {primes, v}];
438     hVectors = {ConstantArray[0, Length[primes]]};
439     Do[hVectors = Union[hVectors, Mod[# + vec[d], 2] & /@ hVectors], {d, radicands}];
440     h = Sort[classOf /@ hVectors];
441     all = Tuples[{0, 1}, Length[primes]];
442     Do[If[Length[cosets] >= $cfg["MaxCosets"], limitHit["MaxCosets"]; Break[]];
443      rep = Sort[classOf /@ (Mod[# + v, 2] & /@ hVectors)];
444      If[! KeyExistsQ[seen, rep], AssociateTo[seen, rep -> True]; AppendTo[cosets, rep]],
445      {v, all}];
446     cosets];
447
448   (* solve (Sum[x_u Sqrt[u]])^2 == rho, rho given as class -> coefficient *)
449   surdSystem[u_List, coeffs_Association] := Module[{vars, cand, sq, basis, eqs, sol},
450     If[u === {} || Length[u] > 16, Return[{}]];
451     bump["CosetSystems"];
452     vars = Table[Unique["RadicalDenest3`Private`coef"], {Length[u]}];
453     cand = vars . Sqrt[u];
454     sq = bounded[Expand[cand^2]];
455     If[sq === $Failed, Return[{}]];
456     basis = Union[Keys[coeffs], DeleteDuplicates[Cases[sq, Power[n_Integer, Rational[1, 2]] :> n,
          {0, Infinity}]]];
457     eqs = Prepend[Table[Coefficient[sq, Sqrt[bb]] == Lookup[coeffs, bb, 0], {bb, DeleteCases[
          basis, 1]}],
458       (sq /. Power[_Integer, Rational[1, 2]] -> 0) == Lookup[coeffs, 1, 0]];
459     sol = bounded[Solve[eqs, vars, Rationals]];
460     (* only fully rational solutions are candidates; Solve may return
461        conditional or parametric solutions, which are not *)
462     If[! ListQ[sol], Return[{}]];
463     sol = Select[sol, ListQ[#] && AllTrue[#, MatchQ[#, _Rule] && rationalQ[Last[#]] &] && Length
          [#] === Length[vars] &];
464     DeleteDuplicates[(cand /. #) & /@ sol]];
465
466   (* integer-relation proposal for one unknown set: an integer vector
```

```
467     (n0, n_u) with n0 Sqrt[rho] + Sum[n_u Sqrt[u]] = 0 gives the candidate
468     -Sum[n_u Sqrt[u]]/n0; it is proposed only when its square reduces to rho,
469     so a spurious relation costs one RootReduce and nothing else *)
470  surdRelation[u_List, rho_] := Module[{sign, vec, rel, cand},
471     If[u === {} || Length[u] > 32, Return[{}]];
472     (* Sign[] of a sum with negative terms can stay unevaluated; the comparisons decide
        numerically *)
473     sign = Which[TrueQ[bounded[rho > 0, "Certificate"]], 1, TrueQ[bounded[rho < 0, "Certificate"
        ]], -1, True, 0];
474     If[sign === 0, Return[{}]];
475     vec = bounded[N[Prepend[Sqrt[u], Sqrt[sign rho]], 80 + 10 Length[u]]];
476     If[! ListQ[vec] || ! AllTrue[vec, NumberQ], Return[{}]];
477     rel = bounded[FindIntegerNullVector[vec]];
478     If[! ListQ[rel] || ! AllTrue[rel, IntegerQ] || First[rel] === 0, Return[{}]];
479     cand = -(Rest[rel] . Sqrt[u])/First[rel];
480     If[sign === -1, cand = I cand];
481     If[TrueQ[bounded[RootReduce[cand^2 - rho], "Certificate"] === 0], {cand, -cand}, {}]];
482
483  (* a summand c Sqrt[r] with rational c and r > 0 as {squarefree radicand, coefficient};
484     the kernel writes Sqrt[6]/2 as Sqrt[3/2], which is Sqrt[6] with coefficient 1/2 *)
485  surdTerm[t_] := Which[
486     rationalQ[t], {1, t},
487     MatchQ[t, Power[_Integer | _Rational, Rational[1, 2]]], surdTerm[{1, t}],
488     MatchQ[t, Times[_?rationalQ, Power[_Integer | _Rational, Rational[1, 2]]]], surdTerm[{t[[1]],
        t[[2]]}],
489     MatchQ[t, {_?rationalQ, Power[_Integer | _Rational, Rational[1, 2]]}],
490      If[t[[2, 1]] <= 0, $Failed,
491       {Numerator[t[[2, 1]]] Denominator[t[[2, 1]]], t[[1]]/Denominator[t[[2, 1]]]}],
492     True, $Failed];
493
494  multiSurdSquareRoots[rho_] := Module[{e, terms, parsed, surds, coeffs, primes, extra, cosets,
       sets, out = {}},
495     e = bounded[Expand[rho]];
496     If[e === $Failed, Return[{}]];
497     terms = If[Head[e] === Plus, List @@ e, {e}];
498     parsed = surdTerm /@ terms;
499     If[MemberQ[parsed, $Failed], Return[{}]];
500     coeffs = Merge[Rule @@@ parsed, Total];
501     surds = Select[Keys[coeffs], # > 1 &];
502     If[Length[surds] < 2 || Length[surds] > 15, Return[{}]];
503     primes = classPrimes[surds];
504     If[primes === {}, Return[{}]];
505     (* the two cheap unknown sets first, then the cosets of the square-class
506        group of the radicands over the primes of the radicands and of the
507        coefficients; the first system with a rational solution wins *)
508     extra = classPrimes[Flatten[{Numerator[#], Denominator[#]} & /@ Select[Values[coeffs],
        rationalQ]]];
509     cosets = cosetBases[surds, Union[primes, extra]];
510     (* stage 1: certified integer-relation proposals, one per coset *)
511     Do[If[expiredQ[], Break[]];
512      out = surdRelation[U, e];
513      If[out =!= {}, Break[]],
514      {U, cosets}];
515     If[out =!= {}, Return[out]];
516     (* stage 2: the rational systems *)
517     sets = DeleteDuplicates[Join[{Union[{1}, primes], Union[{1}, primes, surds]}, cosets]];
518     Do[If[expiredQ[], Break[]];
519      out = surdSystem[U, coeffs];
520      If[out =!= {}, Break[]],
521      {U, sets}];
522     out];
523
524  (* Gaussian square root: Sqrt[a + b I] with a^2 + b^2 a rational square *)
525  gaussianSquareRoots[z_] := Module[{a = Re[z], b = Im[z], n},
526     n = Sqrt[a^2 + b^2];
527     If[rationalQ[n], {Sqrt[(n + a)/2] + I Sign[b] Sqrt[(n - a)/2]}, {}]];
528
529  (* roots of unity as rational powers of -1 *)
530  unity[k_Integer, q_Integer] := (-1)^(Mod[2 k, 2 q]/q);
531
532  (* candidates for rho^(1/q), rho exact, produced by the shape-specific recipes *)
533  rootCandidates[rho_, q_Integer] := Module[{parts, out = {}, sign = 1, r = rho, neg},
```

```
534     If[q === 2 && Head[rho] === Complex, Return[gaussianSquareRoots[rho]]];
535     neg = TrueQ[bounded[rho < 0, "Certificate"]];
536     If[neg, r = -rho];
537     Which[
538      q === 2,
539       parts = quadraticParts[r];
540       If[parts =!= $Failed, out = quadraticSquareRoots[parts]];
541       out = Join[out, honsbeekSquareRoots[r], multiSurdSquareRoots[r]];
542       If[neg, out = I out],
543      OddQ[q] && q >= 3 && q <= $cfg["MaxOddIndex"],
544       parts = quadraticParts[r];
545       If[parts =!= $Failed, out = quadraticOddRoots[parts, q]];
546       If[neg, out = (-1)^(1/q) out]];
547     out];
548
549   (* ------------------------------------------------------------------ *)
550   (* Kummer-linear factors and index reduction                          *)
551   (* ------------------------------------------------------------------ *)
552
553   (* the radicals (and I) occurring in an expression, as an explicit Extension
554      specification: factoring over them keeps roots in radical presentation,
555      whereas Extension -> Automatic may canonicalize them into Root objects *)
556   radicalExtension[e_] := Module[{ext = DeleteDuplicates[Cases[e, Power[_, _Rational], {0,
        Infinity}]]},
557     If[! FreeQ[e, Complex], ext = Prepend[ext, I]];
558     If[ext === {}, Automatic, ext]];
559
560   (* radical presentations of an algebraic number given with Root objects *)
561   radicalForms[z_] := Module[{out = {z}, r},
562     If[FreeQ[z, _Root | _AlgebraicNumber], Return[out]];
563     r = bounded[ToRadicals[z]];
564     If[r =!= $Failed, AppendTo[out, r]];
565     r = bounded[RootReduce[z], "Certificate"];
566     If[r =!= $Failed, r = bounded[ToRadicals[r]]; If[r =!= $Failed, AppendTo[out, r]]];
567     DeleteDuplicates[Select[out, exactQ]]];
568
569   (* roots gamma of x^k == rho lying in the field generated by the radicals of rho *)
570   linearRoots[rho_, k_Integer] := Module[{fl, roots},
571     fl = bounded[FactorList[$x^k - rho, Extension -> radicalExtension[rho]]];
572     If[fl === $Failed || ! ListQ[fl],
573      fl = bounded[FactorList[$x^k - rho, Extension -> Automatic]]];
574     If[fl === $Failed || ! ListQ[fl], Return[{}]];
575     roots = Cases[fl, {f_, _Integer} /; PolynomialQ[f, $x] && Exponent[f, $x] === 1 :>
576        bounded[Together[-Coefficient[f, $x, 0]/Coefficient[f, $x, 1]]]];
577     DeleteDuplicates[Select[Flatten[radicalForms /@ DeleteCases[roots, $Failed]], exactQ]]];
578
579   (* denest a sub-problem recursively under the shared budget *)
580   recurse[sub_] := If[$recursion >= $cfg["MaxRecursion"] || expiredQ[], sub,
581     Block[{$recursion = $recursion + 1, $cfg = Append[$cfg, "MaxTrials" -> Quotient[$cfg["
        MaxTrials"], 4]]},
582      improveNumber[sub]]];
583
584   (* candidates for target == rho^(p/q) from linear factors of x^k - rho, k | q.
585      Every stage receives the incumbent and returns it unchanged when it finds
586      nothing; the reduced presentation gamma^(1/(q/k)) is offered on its own
587      before any recursion, so an index reduction that is already cheaper does
588      not depend on recursive progress. *)
589   kummerCandidates[target_, rho_, p_Integer, q_Integer, incumbent_: Automatic] :=
590     Module[{best = If[incumbent === Automatic, target, incumbent], gammas, k, rest, sub, dens,
        reduced},
591     Do[
592      If[expiredQ[], Break[]];
593      gammas = linearRoots[rho, k];
594      rest = q/k;
595      If[rest === 1,
596       Do[best = acceptWithPolish[bounded[Expand[(gamma unity[l, q])^p]], target, best, "
        LinearFactor"], {gamma, gammas}, {l, 0, q - 1}],
597       (* rho = gamma^k, so target = (gamma zeta_k^j)^(p/rest) zeta_rest^l for some j, l;
598          the distinct values gamma zeta_k^j are each processed once *)
599       dens = DeleteDuplicates[DeleteCases[Flatten[Table[bounded[Expand[gamma unity[j, k]]], {
        gamma, gammas}, {j, 0, k - 1}]], $Failed]];
600       Do[
```

```
601       If[expiredQ[], Break[]];
602       reduced = den^(1/rest);
603       Do[best = acceptWithPolish[bounded[Expand[(reduced unity[1, rest])^p]], target, best, "
          IndexReduction"], {1, 0, rest - 1}];
604       sub = recurse[reduced];
605       If[sub =!= reduced && exactQ[sub],
606        Do[best = acceptWithPolish[bounded[Expand[(sub unity[1, rest])^p]], target, best, "
          IndexReduction/Recursive"], {1, 0, rest - 1}]],
607       {den, dens}]];
608     If[RadicalDepth[best] <= 1, Break[]],
609     {k, Reverse[Rest[Divisors[q]]]}];
610    best];
611
612 (* negative real radicand: rho^(p/q) = (-1)^(p/q) (-rho)^(p/q); the modulus is
613    denested recursively and the phase-separated candidate is offered whether
614    or not the recursion changed the modulus *)
615 negativeRadicandCandidate[target_, rho_, p_Integer, q_Integer, incumbent_: Automatic] :=
616   Module[{best = If[incumbent === Automatic, target, incumbent], sub},
617    If[! TrueQ[bounded[rho < 0, "Certificate"]], Return[best]];
618    sub = recurse[(-rho)^(p/q)];
619    acceptWithPolish[bounded[Expand[(-1)^(p/q) sub]], target, best, "NegativeRadicand"]];
620
621 (* ------------------------------------------------------------------ *)
622 (* the multiplier search (bounded FIFO queue, single admission gate)  *)
623 (* ------------------------------------------------------------------ *)
624
625 reductionIndex[e_] := Module[{d = RadicalDepth[e], nodes, q = 1},
626    nodes = Select[Cases[e, Power[_, _Rational], {0, Infinity}], RadicalDepth[#] === d &];
627    Do[q = LCM[q, Denominator[Last[node]]];
628     If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[$Failed]], {node, nodes}];
629    If[q < 2, $Failed, q]];
630
631 complementaryFactor[Power[b_, r_Rational]] := b^(Mod[-Numerator[r], Denominator[r]]/Denominator
       [r]);
632 complementaryFactor[Complex[0, _]] := -I;
633 complementaryFactor[e_] := Module[{deep = Cases[e, Power[_, _Rational], {0, Infinity}], maxd},
634    If[deep === {}, 1, maxd = Max[RadicalDepth /@ deep];
635     Times @@ (complementaryFactor /@ Select[deep, RadicalDepth[#] == maxd &])]];
636 complementaryMultiplier[term_] := Times @@ (complementaryFactor /@ If[Head[term] === Times,
       List @@ term, {term}]);
637
638 multiplierSearch[target_, initialBest_] := Module[
639    {best = initialBest, q, rho, queue = {}, seen = <||>, admitted = 0, proposed = 0, cursor = 1,
640     cap = $cfg["MultiplierCap"], proposalCap, admit, seeds, terms, m, theta, p, degree, gcd, gd,
         roots,
641     mroot, candidate, disc, primes, j, digits, batch, sol, trials = 0, sinceImprovement = 0,
        bestDegree = Infinity},
642    If[cap === 0 || $cfg["MaxTrials"] === 0 || expiredQ[], Return[best]];
643    q = reductionIndex[target]; If[q === $Failed, Return[best]];
644    rho = bounded[Expand[target^q]]; If[rho === $Failed || ! exactQ[rho], Return[best]];
645    proposalCap = 4 cap;
646    (* the only insertion point: caps admissions, counts proposals, dedups by exact canonical
         value *)
647    admit[value_] := Module[{canonical, key},
648      If[admitted >= cap, limitHit["MultiplierCap"]; Return[False]];
649      If[proposed >= proposalCap, limitHit["MultiplierProposals"]; Return[False]];
650      proposed++; bump["MultipliersProposed"];
651      If[! exactQ[value] || ! smallQ[value], Return[False]];
652      (* the key is the expanded form: algebraically equal multipliers written
653         differently are rare among products of primes and visible radicals, and a
654         RootReduce per proposal would dominate the cost of a trial *)
655      canonical = bounded[Expand[value]];
656      If[canonical === $Failed || TrueQ[canonical == 0], Return[False]];
657      key = ToString[canonical, InputForm];
658      If[KeyExistsQ[seen, key], bump["DuplicateMultipliers"]; Return[False]];
659      AssociateTo[seen, key -> True]; AppendTo[queue, value];
660      admitted++; bump["MultipliersAdmitted"]; True];
661    If[ListQ[$cfg["Multipliers"]],
662     seeds = $cfg["Multipliers"],
663     terms = DeleteCases[Replace[#, {Times[a___, _?rationalQ, b___] :> a*b, _?rationalQ -> 0,
        Complex[a_, b_] :> Sign[b] I}] & /@
```

```
664           If[Head[rho] === Plus, List @@ rho, {rho}], 0];
665       seeds = Join[{1}, DeleteCases[complementaryMultiplier /@ terms, 1], {2, 3, 5}]];
666      Do[If[admitted >= cap || proposed >= proposalCap || expiredQ[], Break[]]; admit[seed], {seed,
           seeds}];
667      While[cursor <= Length[queue] && ! expiredQ[],
668       (* "MaxTrials" bounds the trials spent on this island; "Patience" stops the
669          search when an improvement exists (found by an earlier stage or by this
670          search) and the last "Patience" trials did not improve on it *)
671       If[trials >= $cfg["MaxTrials"], limitHit["MaxTrials"]; Break[]];
672       If[best =!= target && sinceImprovement >= $cfg["Patience"], limitHit["Patience"]; Break[]];
673       m = queue[[cursor]]; cursor++; trials++; sinceImprovement++; bump["Trials"];
674       (* the multiplied radicand keeps its radical presentation: the GCD roots then
675          come out in the radicals of rho, not as opaque algebraic numbers *)
676       theta = bounded[Expand[m rho]];
677       If[theta === $Failed, Continue[]];
678       p = bounded[MinimalPolynomial[theta^(1/q), $x]];
679       If[! validPolynomialQ[p, $x], If[p =!= $Failed, limitHit["MaxDegree"]]; Continue[]];
680       degree = Exponent[p, $x];
681       log["multiplier ", m, "  minpoly degree ", degree];
682       bestDegree = Min[bestDegree, degree];
683       trace["Trial", <|"Multiplier" -> m, "Degree" -> degree, "Index" -> q|>];
684       (* roots of x^q == m rho: linear factors over the radicals of the radicand, then the
685          proper GCD factor with the minimal polynomial of the principal root *)
686       roots = If[m === 1, {}, linearRoots[theta, q]];
687       gcd = bounded[PolynomialGCD[p, $x^q - theta, Extension -> Automatic]];
688       If[validPolynomialQ[gcd, $x],
689        gd = Exponent[gcd, $x];
690        If[gd < q,
691         roots = Join[roots, Which[
692            gd === 1, {-Coefficient[gcd, $x, 0]/Coefficient[gcd, $x, 1]},
693            gd <= $cfg["MaxSolveDegree"],
694             sol = bounded[Solve[gcd == 0, $x]];
695             If[ListQ[sol], Select[$x /. sol, exactQ], {}],
696            True, {}]]]];
697       roots = DeleteDuplicates[Select[Flatten[radicalForms /@ roots], exactQ]];
698       If[roots =!= {},
699        Module[{},
700         mroot = bounded[m^(1/q)];
701         If[mroot =!= $Failed,
702          Do[If[expiredQ[], Break[]];
703           Do[If[expiredQ[], Break[]];
704            candidate = bounded[Expand[z/mroot unity[k, q]]];
705            Module[{before = best},
706             best = acceptWithPolish[candidate, target, best, "MultiplierOrbit"];
707             If[best =!= before, sinceImprovement = 0]];
708            (* a certified but not cheaper candidate (e.g. gamma^(1/3) with gamma in Q(rho))
709               may itself be denestable: try it recursively *)
710            If[candidate =!= $Failed && RadicalDepth[candidate] <= RadicalDepth[target] &&
          candidate =!= target &&
711              certify[candidate, target] === "Equal",
712             Module[{before = best},
713              best = acceptWithPolish[recurse[candidate], target, best, "MultiplierOrbit/Recursive
          "];
714              If[best =!= before, sinceImprovement = 0]]],
715            {k, 0, q - 1}],
716           {z, roots}]]]];
717       If[RadicalDepth[best] <= 1, Break[]];
718       (* discriminant primes of a promising trial (smallest degree seen so far): stream
719          prime powers and a bounded prefix of mixed-radix products *)
720       If[! ListQ[$cfg["Multipliers"]] && degree <= bestDegree && admitted < cap && proposed <
          proposalCap && ! expiredQ[],
721        disc = bounded[Discriminant[p, $x]];
722        If[IntegerQ[disc] && disc =!= 0,
723         primes = bounded[Take[Select[First /@ FactorInteger[Abs[disc]], PrimeQ], UpTo[8]]];
724         If[! ListQ[primes], primes = {}];
725         batch = 0;
726         Do[If[admitted >= cap || proposed >= proposalCap || batch >= $cfg["DiscriminantBatchCap"]
          || expiredQ[], Break[]];
727          Do[If[admitted >= cap || proposed >= proposalCap || batch >= $cfg["DiscriminantBatchCap"
          ] || expiredQ[], Break[]];
728           batch++; admit[m prime^e], {e, 1, q - 1}], {prime, primes}];
729         j = 1;
```

```
730       While[primes =!= {} && j < q^Length[primes] && batch < $cfg["DiscriminantBatchCap"] &&
          admitted < cap && proposed < proposalCap && ! expiredQ[],
731         digits = IntegerDigits[j, q, Length[primes]]; j++; batch++;
732         admit[m Times @@ MapThread[Power, {primes, digits}]]]]]];
733     If[admitted >= cap, limitHit["MultiplierCap"]];
734     If[trials >= $cfg["MaxTrials"], limitHit["MaxTrials"]];
735     best];
736
737 (* ------------------------------------------------------------------ *)
738 (* one exact algebraic island                                         *)
739 (* ------------------------------------------------------------------ *)
740
741 (* whole-island proposals that do not depend on the shape *)
742 genericProposals[target_, incumbent_] := Module[{best = incumbent, r, mp, solver},
743     solver = $cfg["Solver"];
744     If[solver =!= Automatic,
745      r = bounded[Catch[Catch[solver[target], _, $Failed &]]];
746      best = acceptWithPolish[r, target, best, "Solver"]];
747     If[! FreeQ[target, _Root | _AlgebraicNumber],
748      r = bounded[ToRadicals[target]];
749      If[r =!= $Failed && r =!= target, best = acceptWithPolish[recurse[r], target, best, "
          ToRadicals"]]];
750     r = bounded[RootReduce[target], "Certificate"];
751     If[r =!= $Failed,
752      best = accept[r, target, best, "RootReduce"];
753      If[opaqueQ[r] && Head[r] === Root && PolynomialQ[First[r][$x], $x] && Exponent[First[r][$x],
           $x] <= 4,
754       best = acceptWithPolish[bounded[ToRadicals[r]], target, best, "RootReduce/ToRadicals"]]];
755     best = accept[bounded[Simplify[target, Assumptions -> True, TimeConstraint -> 5]], target,
          best, "Simplify"];
756     If[TrueQ[$cfg["Factor"]], best = accept[bounded[Factor[target]], target, best, "Factor"]];
757     best = accept[rationalizeRaw[target], target, best, "Rationalize"];
758     best];
759
760 (* fast paths for a single rational-power node; every stage receives and
761    returns the incumbent *)
762 powerProposals[target_, incumbent_] := Module[{best = incumbent, rho, p, q, root, cands, before
          },
763     If[! MatchQ[target, Power[_, _Rational]], Return[best]];
764     rho = First[target]; p = Numerator[Last[target]]; q = Denominator[Last[target]];
765     If[q > $cfg["MaxRootIndex"], limitHit["MaxRootIndex"]; Return[best]];
766     (* shape-specific recipes for the q-th root, then the integer power p *)
767     before = best;
768     cands = bounded[rootCandidates[rho, q]];
769     If[! ListQ[cands], cands = {}];
770     Do[best = acceptWithPolish[bounded[Expand[c^p]], target, best, "FastPath"], {c, cands}];
771     If[best =!= before, bump["FastPathAccepted"]];
772     If[RadicalDepth[best] <= 1, Return[best]];
773     (* negative real radicand: separate the phase, denest the modulus *)
774     best = negativeRadicandCandidate[target, rho, p, q, best];
775     If[RadicalDepth[best] <= 1, Return[best]];
776     (* roots of rho inside Q(rho), and index reduction through divisors of q *)
777     best = kummerCandidates[target, rho, p, q, best];
778     (* an even index: the square root first, then the remaining index *)
779     If[RadicalDepth[best] >= 2 && EvenQ[q] && q > 2,
780      root = recurse[rho^(1/2)];
781      If[root =!= rho^(1/2) && exactQ[root],
782       best = acceptWithPolish[bounded[Expand[(root^(2/q))^p]], target, best, "EvenIndexSplit"];
783       root = recurse[root^(2/q)];
784       best = acceptWithPolish[bounded[Expand[root^p]], target, best, "EvenIndexSplit/Recursive"
          ]]];
785     best];
786
787 (* the multiplier search is meant for a radical node or a product of radical
788    nodes; a sum island gets the whole-island proposals only *)
789 searchableQ[e_] := MatchQ[e, Power[_, _Rational]] ||
790     (Head[e] === Times && AllTrue[List @@ e, MatchQ[#, _?gaussianQ | Power[_, _Rational]] &]);
791
792 (* results for one island are memoized within the session: the same sub-problem
793    recurs through roots of unity, index reduction and repeated passes *)
794 (* The memo records, for every island visited, the best certified result and
795    the budget of the search that produced it (the multiplier trials and the
```

```
796     recursion levels that were available, and whether the search completed
797     inside the time budget).  A memoized improvement is reused as the incumbent
798     of a later visit.  A memoized negative result is reused only when the
799     earlier search had at least the budget of the current one and completed;
800     otherwise the island is searched again, so a weak recursive search never
801     poisons a later top-level search.  Islands on the current recursion path are
802     not re-entered. *)
803  memoBudget[] := {$cfg["MaxTrials"], $cfg["MaxRecursion"] - $recursion};
804  improveNumber[target_] := Module[{best = target, entry},
805     If[expiredQ[] || ! smallQ[target] || ! exactQ[target], Return[target]];
806     If[RadicalDepth[target] < 2 && FreeQ[target, _Root | _AlgebraicNumber], Return[target]];
807     If[KeyExistsQ[$memo, target],
808      entry = $memo[target]; best = entry["Result"];
809      If[entry["Complete"] && And @@ Thread[entry["Budget"] >= memoBudget[]], Return[best]]];
810     If[KeyExistsQ[$inProgress, target], Return[best]];
811     bump["Islands"];
812     Block[{$inProgress = Append[$inProgress, target -> True]},
813      best = powerProposals[target, best];
814      If[RadicalDepth[best] >= 2 || ! FreeQ[best, _Root | _AlgebraicNumber], best =
        genericProposals[target, best]];
815      If[RadicalDepth[best] >= 2 && $cfg["Solver"] === Automatic && searchableQ[target] && !
        expiredQ[],
816       best = multiplierSearch[target, best]]];
817     AssociateTo[$memo, target -> <|"Result" -> best, "Budget" -> memoBudget[], "Complete" -> !
       expiredQ[]|>];
818     best];
819
820  (* an exact algebraic island: children first (all levels) or outermost
821     radical nodes first (default), then the island as a whole *)
822  (* recombination of an island whose components were rewritten: the proposals
823     are computed from the rewritten form but certified against the original *)
824  combineProposals[target_, incumbent_] := Module[{best = incumbent, r},
825     r = bounded[RootReduce[incumbent], "Certificate"];
826     best = accept[r, target, best, "Combine/RootReduce"];
827     best = accept[bounded[Simplify[incumbent, Assumptions -> True, TimeConstraint -> 5]], target,
         best, "Combine/Simplify"];
828     best = accept[bounded[Together[incumbent]], target, best, "Combine/Together"];
829     best = accept[bounded[Expand[incumbent]], target, best, "Combine/Expand"];
830     best];
831
832  island[e_] := Module[{current = e, rebuilt},
833     If[expiredQ[], Return[e]];
834     If[RadicalDepth[e] < 2 && FreeQ[e, _Root | _AlgebraicNumber], Return[e]];
835     If[MatchQ[e, Power[_, _Rational]],
836      If[TrueQ[$cfg["AllLevels"]],
837       (* the radicand is an island of its own; congruence carries its certificate *)
838       rebuilt = Power[island[First[e]], Last[e]];
839       If[rebuilt =!= e && cheaperQ[rebuilt, e], current = rebuilt]];
840      Return[improveNumber[current]]];
841     If[MemberQ[{Plus, Times}, Head[e]] || MatchQ[e, Power[_, _Integer]],
842      (* components first: each component certified against itself *)
843      rebuilt = If[Head[e] === Power, Power[island[First[e]], Last[e]], Map[island, e]];
844      If[rebuilt =!= e && cheaperQ[rebuilt, e], current = rebuilt];
845      If[RadicalDepth[current] >= 2 || ! FreeQ[current, _Root | _AlgebraicNumber],
846       rebuilt = improveNumber[current];
847       If[rebuilt =!= current, current = rebuilt]];
848      If[current =!= e, current = combineProposals[e, current]];
849      Return[current]];
850     If[opaqueQ[e], Return[improveNumber[e]]];
851     e];
852
853  (* ------------------------------------------------------------------ *)
854  (* host traversal by congruence                                       *)
855  (* ------------------------------------------------------------------ *)
856
857  walk[e_] := Module[{h},
858     If[expiredQ[], Return[e]];
859     If[exactQ[e], Return[island[e]]];
860     If[AtomQ[e], Return[e]];
861     h = Head[e];
862     Which[
863      MemberQ[{List, Plus, Times}, h], Map[walk, e],
```

```
    h === Power && Length[e] === 2 && MatchQ[Last[e], _Integer | _Rational], Power[walk[First[e]], Last[e]],
    True, e]];

(* ------------------------------------------------------------------ *)
(* the session                                                        *)
(* ------------------------------------------------------------------ *)

run[e_, cfg_Association, coreOnly_: False] :=
  Block[{$active = True, $cfg = cfg, $deadline = AbsoluteTime[] + cfg["TimeBudget"],
    $stats = newStats[], $limits = <||>, $trace = {}, $records = {}, $memo = <||>, $inProgress = <||>,
    $recursion = 0, $lastCertificateMethod = "None", $Assumptions = True},
   Module[{committed = {e, Missing["NotComputed"]}, initialCost = Missing["NotComputed"], candidate, candidateCost,
     outcome = Null, started = AbsoluteTime[], passes, status, numericInput = False},
    passes = If[TrueQ[cfg["AllLevels"]] && ! coreOnly, cfg["MaxPasses"], 1];
    If[TrueQ[cfg["TimeBudget"] == 0], limitHit["TimeBudget"],
     (* the classification of the input and every cost computation are inside the region *)
     outcome = TimeConstrained[
       MemoryConstrained[
        Which[
         ! FreeQ[e, _Real | _Complex?InexactNumberQ], limitHit["InexactInput"],
         coreOnly && ! exactQ[e], limitHit["NotExactAlgebraic"],
         True,
         numericInput = exactQ[e];
         initialCost = RadicalCost[e]; committed = {e, initialCost};
         Do[
          If[expiredQ[], Break[]];
          candidate = If[coreOnly, improveNumber[First[committed]], walk[First[committed]]];
          bump["PassesCompleted"];
          If[candidate === First[committed], Break[]];
          candidateCost = RadicalCost[candidate];
          If[Order[candidateCost, Last[committed]] =!= 1, Break[]];
          (* whole-input certificate when the whole input is a number; hosts rely on congruence *)
          If[numericInput && certify[candidate, e] =!= "Equal", Break[]];
          (* the pass and its cost are published together *)
          committed = {candidate, candidateCost};
          If[pass === passes && passes > 1, limitHit["MaxPasses"]],
          {pass, passes}]],
        cfg["MemoryBudget"], memoryStopped],
       cfg["TimeBudget"], timeStopped];
     If[outcome === memoryStopped, limitHit["MemoryBudget"]];
     If[outcome === timeStopped, limitHit["TimeBudget"]]];
    status = Which[
      outcome === timeStopped, "Timeout",
      outcome === memoryStopped, "MemoryLimit",
      TrueQ[cfg["TimeBudget"] == 0], "Disabled",
      First[committed] === e, "Unchanged",
      True, "Improved"];
    <|"Result" -> First[committed], "Status" -> status, "ResultChanged" -> (First[committed] =!= e),
     "Limits" -> Keys[$limits],
     "InitialCost" -> initialCost, "FinalCost" -> Last[committed],
     "Statistics" -> $stats, "Certificates" -> $records,
     "CertificateKind" -> "Kernel-checked accepted proposals (Before, After, Method, EqualityMethod); a bounded sample, not a proof chain of the final result",
     "CertificatesTruncated" -> (Lookup[$stats, "CandidatesAccepted", 0] > Length[$records]),
     "LimitsMeaning" -> "Guards that were triggered or caps that were reached; not impossibility certificates",
     "ElapsedSeconds" -> AbsoluteTime[] - started, "Options" -> cfg,
     "Trace" -> $trace, "CompletenessClaim" -> False|>]];

invoke[e_, head_Symbol, rules_List, report_, core_] := Module[{cfg, result},
   cfg = resolveOptions[head, rules];
   If[FailureQ[cfg], Return[cfg]];
   result = run[e, cfg, core];
   If[TrueQ[report], result, result["Result"]]];

(* every argument sequence reaches the option validator, so a malformed call
   such as Strad[e, 17] returns a Failure instead of staying unevaluated *)
```

```
Strad[e_, all : (True | False), args___] := invoke[e, Strad, {"AllLevels" -> all, args}, False, False];
Strad[e_, args___] := invoke[e, Strad, {args}, False, False];
DenestRadicals[e_, all : (True | False), args___] := invoke[e, DenestRadicals, {"AllLevels" -> all, args}, False, False];
DenestRadicals[e_, args___] := invoke[e, DenestRadicals, {args}, False, False];
DenestCore[e_, args___] := invoke[e, DenestCore, {args}, False, True];
DenestReport[e_, all : (True | False), args___] := invoke[e, DenestReport, {"AllLevels" -> all, args}, True, False];
DenestReport[e_, args___] := invoke[e, DenestReport, {args}, True, False];
Strad[] := Failure["InvalidArguments", <|"MessageTemplate" -> "An expression is required."|>];
DenestRadicals[] := Failure["InvalidArguments", <|"MessageTemplate" -> "An expression is required."|>];
DenestCore[] := Failure["InvalidArguments", <|"MessageTemplate" -> "An expression is required."|>];
DenestReport[] := Failure["InvalidArguments", <|"MessageTemplate" -> "An expression is required."|>];

End[];
EndPackage[];
```

B Files of this directory

File	Purpose
`unified_analysis_C.tex`, `.pdf`	This document and its source; the appendix reads `../../corrected/StradFixed3.wl`.
`tables/*.tex`	Result tables generated by `harness/export_tables3.wl`.
`harness/runner3.wl`	Runner for the battery (loads the package, captures values, messages, times, depths).
`harness/exp_battery3.wl`	The unified-A/B battery and the round-3 cases on the new version.
`harness/cases_new.wl`	The round-3 cases (group N), shared by both versions' runs.
`harness/exp_new2.wl`	The round-3 cases on `StradFixed2.wl` (uses unified-B's `runner2.wl`).
`harness/exp_fuzz3.wl`	The fourteen randomized families, same seed.
`harness/probe_gaps3_1..4.wl`	The probing scripts of `KNOWN_GAPS.md` on the new version.
`harness/probe_reviews2.wl`	The reviews' probes against `StradFixed2.wl`, executed.
`harness/differential3.wl`	Review-8's differential corpus on both versions.
`harness/repro_fuzz_slow.wl`	Replays the randomized inputs and times the slow ones of chosen families (used to find C22).
`harness/run_review_suite3.wl`	Runs a review's own suite against its own proposal (argument 7, 8 or 9).
`harness/export_tables3.wl`	Generates `tables/` from the recorded rows.
`harness/run_all.sh`	The sequential chain that produced `logs/`.
`tests/StradFixed3.wlt`	The regression suite (230 tests).
`tests/run_tests.wls`	Its fresh-kernel runner.

File	Purpose
`logs/*.txt`	Raw transcripts of every run reported here.